

Unified Accelerator Fabric (UAF)

Engineering Specification for Implementation (UAF-SPEC-001)

UAF Working Group

Version 2.1 — Draft for Approval

Contents

Document Control	3
Breaking changes from v2.0	3
1 Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Design principles	5
1.4 Normative language	5
1.5 References	6
1.6 Approval record	6
2 Terminology and Naming	6
2.1 Backend codewords	7
3 System Architecture	7
3.1 Layer diagram	7
3.2 Wire profiles	8
3.3 CXL generation	8
3.4 Compile-time configuration (uaf_config.h)	9
4 Core Data Structures and ABI	10
4.1 ABI rules	10
4.2 Handles and the memory region	10
4.3 Enumerations	11
4.4 Work requests and completions	12
4.5 Queue pair creation and modification	13
4.6 Memory registration	14
4.7 Return conventions	14
4.8 Device capability query	15
4.9 Threading	15
4.10 Backend loading	15
5 RMT Protocol — Profile B (UAF-RMT over UDP)	16
5.1 Packet format	16
5.2 Field definitions	16
5.3 Opcode semantics	18
5.4 Atomics	19
5.5 Connection management	19
5.6 Reliability	21

5.7	Error handling	22
5.8	Test vectors	22
5.9	Congestion control	23
6	DST Protocol	24
6.1	Structure of this section	24
6.2	Command mapping	24
6.3	Completion ring	25
6.4	Storage queue state machine	26
6.5	Command identifiers	26
6.6	Submission ring memory type	26
7	Platform Backend: UAF-N (NVIDIA)	26
7.1	Profile and capabilities	27
7.2	Device discovery	27
7.3	Memory registration	27
7.4	DST via cuFile	28
8	Platform Backend: UAF-S (Snapdragon, Linux)	29
8.1	Memory registration	29
8.2	Send path	31
9	Platform Backend: UAF-D (Dragonfly)	31
9.1	Memory registration	32
9.2	Send ring over CXL.mem	33
9.3	DST over a CXL-mapped NVMe BAR	33
9.4	Device-visible ordering	34
10	State Machines, Memory, and Rings	35
10.1	Queue-pair state machine	35
10.2	Key encoding	36
10.3	Ring discipline	36
11	Security	36
11.1	Threat model and mitigations	36
11.2	Key management	37
12	Performance and Conformance	38
12.1	Measurement method	38
12.2	Targets	38
12.3	Conformance suite	40
13	Build System	41
14	Appendices	42
14.1	Appendix A: Public header (uaf_core.h)	42
14.2	Appendix B: Reference application (rdma_write_demo.c)	44
14.3	Appendix C: Requirement index	46
14.4	Appendix D: Open issues	49
14.5	Appendix E: Disposition of the v2.0 reviews	49

Document Control

Version	Date	Author	Changes
0.9	—	UAF WG	Initial draft
1.0	2026-01-01	UAF WG	Full engineering specification, PDF-ready
2.0	2026-09-29	UAF WG	CXL/NVMe doorbell ordering; userspace dma-heap + FastRPC SMMU mapping; C ABI completed; 32-bit rkey; 16-byte packed DST CQE
2.1	2026-09-29	UAF WG	Closes both v2.0 reviews. One data plane per backend and two declared wire profiles; message framing, atomic responses, CM packet layouts; defined poll return convention; uaf_query_device; in-place registration; one CSPRNG key algorithm; NVMe mapping replaces the unused pseudo-SQE; corrected reliability and performance figures; RFC 2119 language with numbered requirements and an executable conformance suite

Classification: Engineering Internal

Status: Draft for Approval. This revision is **not** approved for implementation. Approval requires the sign-off block in Section 1.6 to be completed and the open issues in Appendix D to be closed or accepted.

Target platforms: NVIDIA (H100/B200/GB200), Qualcomm Snapdragon (X Elite, 8 Gen 3–4), Qualcomm Dragonfly (C1000).

Breaking changes from v2.0

v2.0 was never implemented, so these changes are made cleanly rather than carried as compatibility debt. Every one of them closes a defect that made the prior text unimplementable or ambiguous.

Change	Reason
uaf_poll_cq / uaf_storage_poll return a count, not UAF_OK	The convention was undefined and the two candidate readings are incompatible once max > 1
UAF_ERR_CQ_EMPTY (-8) retired and its value reserved	An empty queue is 0 entries, not an error; no other error code is renumbered
Wire header re-laid out; lkey_rsvd removed, msg_id, msg_len, seg_off, data_crc32c added	A multi-packet message could not be framed and the payload was outside every integrity check
UAF_WIRE_MTU_DEFAULT now counts the 64-byte header	v2.0's maximum packet was 4160 bytes and did not fit a 4 KiB path MTU
UAF_WR_RECV removed from the wire opcode table	It is a local buffer post, not a packet
No enum or size_t field in any ABI struct; every struct pinned by _Static_assert	enum width is implementation-defined; size_t differs between ILP32 and LP64 Snapdragon userspace
uaf_mr.pal_priv removed from the public struct	It froze every backend's internals into the application ABI
dma_buf_fd removed from uaf_conn_info	A file descriptor cannot be carried in a connection exchange
uaf_storage_cqe no longer packed; gains a phase tag	packed dropped alignment to 1; without a phase tag a zeroed slot is indistinguishable from UAF_OK
uaf_storage_close(sq, cr) takes both objects	v2.0 closed only the submission queue
Section 6.1's 64-byte "UAF SQE" deleted	No backend ever filled it; DST is specified as an NVMe mapping
uaf_create_qp takes struct uaf_qp_init_attr;	Depth, SGE limits and queue-pair type were unspecified, and which fields a transition read was unstated
uaf_modify_qp takes an attribute mask	

1 Introduction

1.1 Purpose

This document specifies the **Unified Accelerator Fabric (UAF)**, a portable C11 framework that provides two services across heterogeneous accelerator platforms:

- **Remote Memory Transport (RMT)** — direct memory access between compute nodes without CPU mediation in the data path.
- **Direct Storage Transport (DST)** — accelerator-initiated storage I/O to NVMe devices.

1.2 Scope

UAF specifies three platform backends.

Backend	Silicon	RMT data plane	DST data plane
UAF-N	NVIDIA H100/B200/GB200 with ConnectX-7/8	InfiniBand Verbs RC over IB or RoCEv2	cuFile (GPUDirect Storage)
UAF-S	Snapdragon X Elite, 8 Gen 3-4 (Linux)	UAF-RMT over UDP; PCIe P2P or FastRPC for local paths	NVMe via io_uring with dma-heap buffers
UAF-D	Dragonfly C1000	CXL.mem load/store with a software ring	NVMe over a CXL-mapped BAR

Non-goals

[R-1.2-001] The following are explicitly **out of scope** for v2.1. An implementation **MUST NOT** claim conformance on the basis of any of them.

- **Windows.** DirectStorage 1.4 is a Windows C++ API and was listed in v2.0's UAF-S row beside Linux dma-heap and FastRPC, which are not the same operating environment. A Windows binding SHALL be a separate document, UAF-SPEC-002. This document specifies Linux only.
- **Unreliable and multicast service.** Only the reliable connected queue pair (UAF_QPT_RC) is required. UAF_QPT_UD is OPTIONAL and capability-gated.
- **Shared receive queues, memory windows, on-demand paging, and re-registration.**
- **Interrupt-driven completion.** UAF_CAP_CQ_EVENTS is declared so a backend can advertise it, and the event API is deferred to v2.2. All required paths are polled.
- **An asynchronous event channel** for link-down and queue-pair fatal notification. Deferred to v2.2; see Appendix D, issue OI-3.
- **A fabric manager, subnet manager, or address resolution service.** Connection information reaches a peer by means outside this specification.
- **DOCA GPUNetIO.** v2.0's scope table named it and mapped it to nothing. It is a possible future path for a UAF-N implementation to speak Profile B; see Appendix D, issue OI-2.

Operating environment

[R-1.2-002] An implementation SHALL support Linux 5.15 or later on x86_64 or aarch64. UAF-D additionally requires a kernel with device-DAX (CONFIG_DEV_DAX) and CXL support (CONFIG_CXL_BUS). UAF-S requires CONFIG_DMABUF_HEAPS and a FastRPC-capable DSP stack.

[R-1.2-003] **One process, one backend.** A process SHALL load exactly one backend library. The core library resolves a single backend at uaf_init() time (Section 4.10); it **MUST NOT** link more than one, and the behaviour of a process that loads two is undefined.

1.3 Design principles

1. **Single C11 ABI.** An application compiles once against uaf_core.h and links against one backend (libuaf_n.so, libuaf_s.so, or libuaf_d.so).
2. **No CPU in the data path where the hardware permits.** UAF-N and UAF-D keep the host CPU out of the steady-state data path. UAF-S uses zero-copy SMMU-mapped rings or DSP offloads and does not make this claim for its UDP path.
3. **Zero-copy by default.** Every buffer path uses in-place registration of caller memory, or memory the library itself allocated. No path in this specification copies a payload in order to register it.
4. **Fail-explicit.** Every function reports a typed error. No silent degradation, no silent truncation, and no silent relocation.
5. **The portable surface is the API, not the packet.** This replaces v2.0's claim that one wire format spans all three backends, which could not be true while UAF-N was also offloaded to ConnectX hardware. An implementation declares a **wire profile** (Section 3.2); interoperability is defined *within* a profile, and Profile B over UDP is the common denominator available to every backend.

1.4 Normative language

[R-1.4-001] The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119.

[R-1.4-002] Requirements are identified as [R-*<section>-<nnn>*] and are indexed in Appendix C. A conformance claim SHALL cite the requirement identifiers it satisfies.

[R-1.4-003] **Normative versus informative.** Tables of field offsets, requirement statements, and the headers reproduced in Appendix A are NORMATIVE. Every other code listing in Sections 7 through 9 is INFORMATIVE: it illustrates one way to satisfy the requirements and is not itself a requirement. v2.0 placed its backend listings in the body with no such marking, which made each of their defects arguably binding.

1.5 References

- RFC 2119, *Key words for use in RFCs to Indicate Requirement Levels*
- RFC 3720 Appendix B.4, *CRC32C (Castagnoli) definition and test vectors*
- InfiniBand Architecture Specification Vol. 1, Release 1.4
- Compute Express Link Specification, Revision 3.1 (PCIe 6.0 PHY, 64 GT/s)
- Compute Express Link Specification, Revision 4.0 (PCIe 7.0 PHY, 128 GT/s)
- NVM Express Base Specification, Revision 2.0d
- Microsoft DirectStorage 1.4 API Reference (out of scope; see Section 1.2)
- Qualcomm FastRPC User Guide (80-NH713-1)
- Linux rdma-core v50, libibverbs, `<linux/dma-heap.h>`, `<linux/dma-buf.h>`

1.6 Approval record

[R-1.6-001] This document SHALL NOT carry an “Approved for Implementation” status until every row below is complete. v2.0 carried that status with no approver, no review record and no open-issues list.

Role	Name	Date	Signature
Specification editor			
RMT protocol reviewer			
DST protocol reviewer			
UAF-N implementation owner			
UAF-S implementation owner			
UAF-D implementation owner			
Security reviewer			
Performance and conformance reviewer			

Review record. v2.0 received two independent engineering reviews, both dated 2026-09-29. Their findings are dispositioned in Appendix E. Open items are carried in Appendix D.

2 Terminology and Naming

Term	Definition
RMT	Remote Memory Transport — RDMA-equivalent service
DST	Direct Storage Transport — accelerator-initiated NVMe I/O
UAF	Unified Accelerator Fabric
PAL	Platform Abstraction Layer

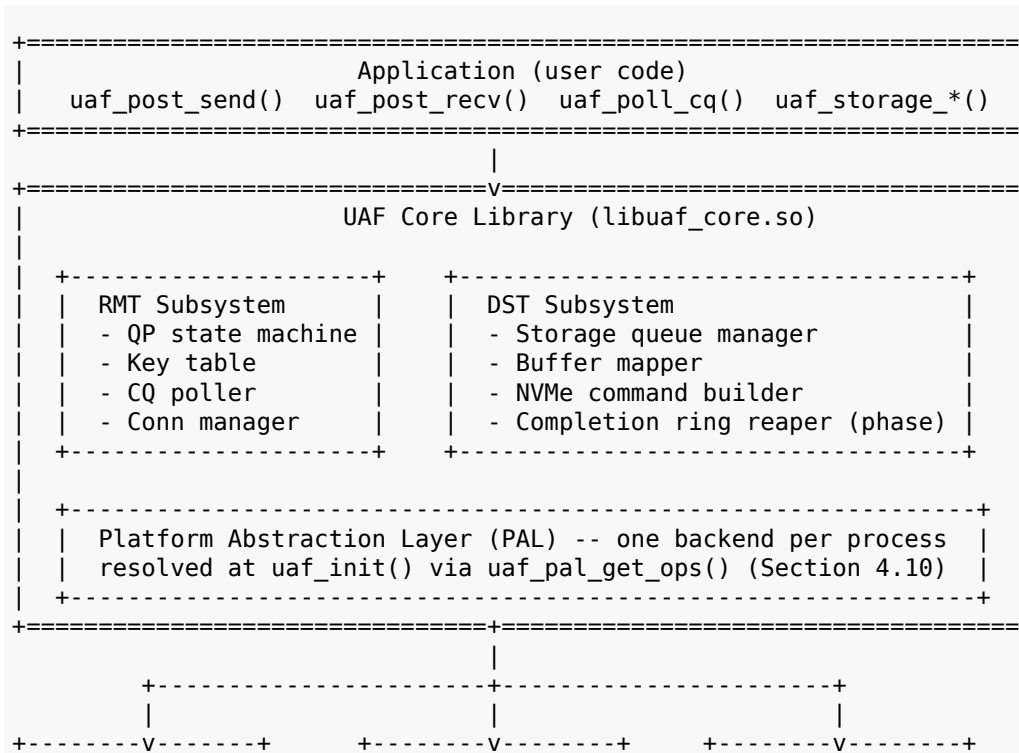
Term	Definition
QP	Queue Pair (send and receive queues)
CQ	Completion Queue (RMT)
STQ	Storage Queue (DST submission). v2.0 called both the send queue and the storage queue "SQ", in adjacent sections of one backend
CR	Completion Ring (DST)
SGE	Scatter-Gather Element
WR	Work Request
WC	Work Completion
CQE	Completion Queue Entry
Segment	The payload of one UAF-RMT packet; a message is one or more segments
Profile A / Profile B	The two wire profiles of Section 3.2

2.1 Backend codewords

Codeword	Platform
UAF-N	NVIDIA (ConnectX with Hopper or Blackwell)
UAF-S	Qualcomm Snapdragon (Adreno with Hexagon DSP), Linux
UAF-D	Qualcomm Dragonfly (C1000 CXL accelerator)

3 System Architecture

3.1 Layer diagram



UAF-N libuaf_n.so Profile A	UAF-S libuaf_s.so Profile B	UAF-D libuaf_d.so Profile B
ibv RC verbs cuFile	UAF-RMT / UDP io_uring + NVMe dma-heap, SMMU	CXL.mem ring NVMe over BAR device-DAX
ConnectX-7/8 H100/B200 GPU	PCIe / SMMU Adreno / Hexagon	CXL Root Port NVMe over CXL

3.2 Wire profiles

[R-3.2-001] An implementation SHALL declare its wire profiles in `uaf_query_device()`'s `wire_profiles` field. Two profiles are defined.

Profile	Bit	Wire format	Reliability and congestion control
A (UAF_PROFILE_IBV)	1 << 0	IB transport headers (BTH, RETH) over InfiniBand or RoCEv2	Hardware: IB RC sequencing, ACK and retransmission; DCQCN offloaded to the adapter
B (UAF_PROFILE_UAFR)	1 << 1	The 64-byte UAFR header of Section 5 over UDP	Software: Sections 5.6 and 5.9

[R-3.2-002] A UAF-N implementation SHALL declare Profile A. It MUST NOT claim that a ConnectX adapter emits the UAFR header: the adapter emits IB transport headers, and the hardware reliable-connected engine supplies its own PSN, ACK and retransmission. It MAY additionally declare Profile B, and if it does, the Profile B path SHALL NOT claim UAF CAP HW CC.

[R-3.2-003] A UAF-S implementation SHALL declare Profile B.

[R-3.2-004] A UAF-D implementation SHALL declare Profile B for node-to-node traffic. Its intra-host path is CXL.mem load/store through a software ring (Section 9.2) and carries no packet header at all; CXL.mem cannot carry a UAFR packet.

[R-3.2-005] Two implementations interoperate if and only if they share a profile. A conformance claim SHALL name the profile it was tested under.

Why this changed. v2.0 principle 5 asserted that UAF-RMT packets interoperate across all three backends over RoCEv2/UDP, while Section 5.6 simultaneously offloaded UAF-N congestion control to ConnectX hardware and Section 7.1 registered memory with `ibv_reg_mr`. Those cannot all hold: hardware offload means the IB transport is on the wire, and the `0x55414652` header is not. Neither can CXL.mem carry a UDP packet, nor is DMA-BUF a fabric. The portable surface is the C API.

3.3 CXL generation

[R-3.3-001] The UAF-D baseline SHALL be **CXL 3.1 on a PCIe 6.0 PHY at 64 GT/s**. An implementation targeting a PCIe 7.0 PHY at 128 GT/s SHALL cite **CXL 4.0** and declare it explicitly.

because CXL 3.1 does not define that PHY. v2.0's scope table paired "CXL 3.1" with "PCIe Gen7", which is one generation apart, and its performance table then quoted a PCIe 7.0 line rate against a CXL 3.1 link.

[R-3.3-002] UAF_ENABLE_CXL_CACHE defaults to 0, and a build with it clear SHALL NOT advertise CXL.cache in any capability or document. v2.0's scope table listed CXL.cache as a UAF-D transport while the build flag that enables it defaulted to off. Where host-coherent atomics are required, CXL.cache is REQUIRED; see [R-5.4-006].

3.4 Compile-time configuration (uaf_config.h)

[R-3.4-001] A feature flag in uaf_config.h MUST NOT change the size or layout of any ABI struct. Every such struct is pinned by a _Static_assert (Section 4.1), so a build that violates this fails to compile.

[R-3.4-002] The resource limits in uaf_config.h are build-time defaults. An application MUST obtain the limits it relies on from uaf_query_device(). v2.0 baked UAF_MAX_SGE, UAF_MAX_INLINE and the queue depths into the application with no runtime query at all, so an application and a library built from different headers would disagree silently.

```
#define UAF_VERSION_MAJOR      2
#define UAF_VERSION_MINOR     1
#define UAF_VERSION_PATCH     0

/* Resource limits (defaults; query at runtime) */
#define UAF_MAX_QP             4096
#define UAF_MAX_CQ             4096
#define UAF_MAX_MR             65536
#define UAF_MAX_SGE            16
#define UAF_MAX_INLINE         256
#define UAF_CQ_DEPTH_DEFAULT   1024
#define UAF_SQ_DEPTH_DEFAULT   256
#define UAF_IS_POW2(x)         (((x) != 0u) && (((x) & ((x) - 1u)) == 0u))

/* Wire protocol (normative) */
#define UAF_WIRE_MAGIC          0x55414652U /* "UAFR" */
#define UAF_WIRE_VERSION        2
#define UAF_WIRE_HDR_SIZE      64

/* PATH MTU: counts the 64-byte header. v2.0 counted payload only, so a
 * maximum packet was 4160 bytes and did not fit a 4 KiB path MTU. */
#define UAF_WIRE_MTU_DEFAULT    4096
#define UAF_WIRE_MAX_SEG(mtu)  ((uint32_t)((mtu) - UAF_WIRE_HDR_SIZE))
#define UAF_WIRE_SEG_DEFAULT    UAF_WIRE_MAX_SEG(UAF_WIRE_MTU_DEFAULT) /* 4032 */

/* Reliability (Section 5.6) */
#define UAF_RTO_BASE_NS         50000ULL /* 50 us */
#define UAF_RTO_MAX_NS          2000000ULL /* 2 ms */
#define UAF_RETRY_MAX           8
#define UAF_WINDOW_DEFAULT      256 /* segments */

/* Security (Section 11) */
#define UAF_RKEY_FAIL_MAX       16
#define UAF_CM_NONCE_BYTES      8
#define UAF_CM_MAC_BYTES        16
#define UAF_CM_REPLAY_WINDOW_NS 2000000000ULL

#define UAF_CQE_ALIGN           16
```

4 Core Data Structures and ABI

4.1 ABI rules

[R-4.1-001] Every struct that crosses the library boundary SHALL use fixed-width integer types only. No enum field and no `size_t` field is permitted. In C11 an enum's size and signedness are implementation-defined and diverge under `-fshort-enums` and across compilers; `size_t` differs between ILP32 and LP64 Snapdragon userspace. v2.0 used both, in `uaf_wr.opcode`, `uaf_wc.status`, `uaf_qp_attr.qp_state` and `uaf_storage_cmd.buf_len`.

[R-4.1-002] Every such struct SHALL be pinned by a `_Static_assert` on its size in the public header, so layout drift is a compile error.

[R-4.1-003] A struct an application passes in that the library may later extend SHALL carry `struct_size` as its first field, set by the caller. The library SHALL fill only the prefix it understands and SHALL rewrite `struct_size` with what it filled.

[R-4.1-004] No struct in `uaf_types.h` is a wire format. Wire formats are byte arrays at fixed offsets, defined in Sections 5.2 and 5.5.

[R-4.1-005] A public struct MUST NOT expose backend-private state. v2.0's `uaf_mr.pal_priv` put a backend pointer in the application ABI and froze every backend's internals. A backend SHALL instead embed `uaf_mr_t` as the first member of its own private struct and recover it by cast.

The pinned sizes, all verified by conformance case C-1:

Struct	Size (bytes)	Alignment
<code>uaf_mr_t</code>	48	8
<code>struct uaf_sge</code>	16	8
<code>struct uaf_wr</code>	64	8
<code>struct uaf_wc</code>	40	8
<code>struct uaf_qp_init_attr</code>	32	4
<code>struct uaf_qp_attr</code>	80	8
<code>struct uaf_device_attr</code>	64	8
<code>struct uaf_conn_info</code>	64	8
<code>struct uaf_peer_info</code>	40	8
<code>struct uaf_storage_cmd</code>	56	8
<code>struct uaf_storage_cqe</code>	16	4

4.2 Handles and the memory region

```
typedef struct uaf_device uaf_device_t;
typedef struct uaf_domain uaf_domain_t;
typedef struct uaf_qp uaf_qp_t;
typedef struct uaf_cq uaf_cq_t;
typedef struct uaf_sq uaf_sq_t; /* storage queue (STQ) */
typedef struct uaf_cr uaf_cr_t; /* DST completion ring */

/* pal_priv is deliberately absent; see [R-4.1-005]. A backend does:
 * struct uaf_s_mr { uaf_mr_t pub; int dma_buf_fd; ... };
 * #define uaf_mr_priv(m) ((struct uaf_s_mr *) (m))
 */
```

```

typedef struct uaf_mr {
    uint32_t    struct_size;
    uint32_t    lkey;
    uint32_t    rkey;
    uint32_t    reserved0;
    uaf_domain_t *pd;
    void        *addr;
    uint64_t    length;
    uint64_t    flags;
} uaf_mr_t;
_Static_assert(sizeof(uaf_mr_t) == 48, "uaf_mr layout is ABI");

```

4.3 Enumerations

```

enum uaf_qp_state {
    UAF_QPS_RESET = 0, UAF_QPS_INIT = 1, UAF_QPS_RTR = 2, UAF_QPS_RTS = 3,
    UAF_QPS_SQD   = 4, UAF_QPS_SQE   = 5, UAF_QPS_ERR = 6,
};

enum uaf_wr_opcode {                /* LOCAL work request opcodes */
    UAF_WR_SEND           = 0x01,
    UAF_WR_SEND_INLINE    = 0x02,
    UAF_WR_RECV           = 0x03, /* local post only; NOT a wire opcode */
    UAF_WR_RDMA_READ      = 0x04,
    UAF_WR_RDMA_WRITE     = 0x05,
    UAF_WR_RDMA_WRITE_IMM = 0x06,
    UAF_WR_ATOMIC_CMP_SWP = 0x07,
    UAF_WR_ATOMIC_FETCH_ADD = 0x08,
    UAF_WR_STORAGE_READ   = 0x10,
    UAF_WR_STORAGE_WRITE  = 0x11,
    UAF_WR_STORAGE_FLUSH  = 0x12,
};

/* Wire-only opcodes. v2.0 used 0x80..0x91 on the wire and defined them in
 * no enum, so two implementations had nothing to agree on. */
enum uaf_wire_opcode {
    UAF_OP_CM_REQ  = 0x80, UAF_OP_CM_REP = 0x81,
    UAF_OP_CM_RTU  = 0x82, UAF_OP_CM_REJ = 0x83,
    UAF_OP_RDMA_READ_RESP = 0x84,
    UAF_OP_ATOMIC_ACK    = 0x88,
    UAF_OP_ACK           = 0x90, UAF_OP_NAK    = 0x91,
};

enum uaf_error {
    UAF_OK                = 0,
    UAF_ERR_INVALID       = -1, UAF_ERR_NOMEM        = -2,
    UAF_ERR_NODEV         = -3, UAF_ERR_PERM         = -4,
    UAF_ERR_BUSY          = -5, UAF_ERR_TIMEOUT      = -6,
    UAF_ERR_QP_STATE      = -7,
    UAF_ERR_RESERVED_8    = -8, /* was UAF_ERR_CQ_EMPTY; retired */
    UAF_ERR_MR_FAULT      = -9, UAF_ERR_NOT_SUPPORTED = -10,
    UAF_ERR_PROTO         = -11, UAF_ERR_CRC         = -12,
    UAF_ERR_REMOTE        = -13,
    UAF_ERR_RKEY          = -14, /* unknown, or not bound to this QP */
    UAF_ERR_AUTH          = -15, /* CM authentication or replay check */
    UAF_ERR_RELOCATED     = -16, /* cannot register in place */
};

```

```

};

enum uaf_wire_profile { UAF_PROFILE_IBV = 1u << 0, UAF_PROFILE_UAFR = 1u << 1 };

enum uaf_device_cap {
    UAF_CAP_ATOMICS      = 1ull << 0, UAF_CAP_DST          = 1ull << 1,
    UAF_CAP_GPU_MR       = 1ull << 2, UAF_CAP_CXL_MR       = 1ull << 3,
    UAF_CAP_DMABUF_MR    = 1ull << 4, UAF_CAP_CQ_EVENTS    = 1ull << 5,
    UAF_CAP_REG_ANY_MEM  = 1ull << 6, UAF_CAP_SYNC_FAULT   = 1ull << 7,
    UAF_CAP_HW_CC        = 1ull << 8,
};

```

4.4 Work requests and completions

```

struct uaf_sge { uint64_t addr; uint32_t length; uint32_t lkey; };

struct uaf_wr {
    uint64_t      wr_id;
    uint32_t      opcode;      /* enum uaf_wr_opcode */
    uint32_t      num_sge;
    struct uaf_sge *sge;
    uint32_t      send_flags; /* enum uaf_send_flags */
    uint32_t      imm_data;
    uint64_t      remote_addr;
    uint32_t      rkey;
    uint32_t      reserved0;
    uint64_t      compare_add;
    uint64_t      swap;
};

struct uaf_wc {
    uint64_t      wr_id;
    uint32_t      opcode;
    int32_t       status;      /* enum uaf_error */
    uint32_t      byte_len;
    uint32_t      qp_num;
    uint32_t      src_qp;
    uint32_t      imm_data;
    uint32_t      wc_flags;    /* UAF_WC_* */
    uint32_t      reserved0;
};

#define UAF_WC_WITH_IMM      (1U << 0) /* imm_data is valid */
#define UAF_WC_SOLICITED    (1U << 1)
#define UAF_WC_ATOMIC_ORIG  (1U << 2) /* SGE 0 holds the pre-operation value */

```

[R-4.4-001] imm_data in a work completion is valid if and only if UAF_WC_WITH_IMM is set. v2.0 delivered imm_data with no way to know whether it was present.

[R-4.4-002] **Ownership at post time.** uaf_post_send() and uaf_post_recv() SHALL copy the work request *and* the num_sge elements of the SGE array before returning. The caller MAY reuse or free both immediately. The buffers the SGEs point to remain owned by the caller until the corresponding completion is reaped. v2.0 never stated this, and its UAF-S sample copied the work request while leaving sge pointing at caller memory.

[R-4.4-003] num_sge SHALL be at most the max_send_sge or max_recv_sge agreed at uaf_create_qp(), which SHALL be at most uaf_query_device()'s max_sge. A larger value SHALL

fail with UAF_ERR_INVALID.

[R-4.4-004] **Gather is local.** Under Profile B the wire carries one remote_va and one contiguous byte stream; a sender walks its SGE list and emits that stream. The wire carries no SGE list. Under Profile A the adapter performs gather natively. An application observes the same semantics either way.

4.5 Queue pair creation and modification

```
enum uaf_qp_type { UAF_QPT_RC = 1, UAF_QPT_UD = 2 /* OPTIONAL */ };

struct uaf_qp_init_attr {
    uint32_t struct_size;
    uint32_t qp_type;
    uint32_t max_send_wr;      /* MUST be a power of two */
    uint32_t max_recv_wr;     /* MUST be a power of two */
    uint32_t max_send_sge;
    uint32_t max_recv_sge;
    uint32_t max_inline_data;
    uint32_t sq_sig_all;      /* 0: only UAF_SEND_SIGNED WRs complete */
};

enum uaf_qp_attr_mask {
    UAF_QP_STATE = 1u<<0, UAF_QP_SQ_PSN = 1u<<1,
    UAF_QP_RQ_PSN = 1u<<2, UAF_QP_DEST_QPN = 1u<<3,
    UAF_QP_PATH_MTU = 1u<<4, UAF_QP_AV_GID = 1u<<5,
    UAF_QP_AV_LID = 1u<<6, UAF_QP_AV_UDP = 1u<<7,
    UAF_QP_AV_CXL = 1u<<8, UAF_QP_MAX_RD_ATOMIC = 1u<<9,
    UAF_QP_MAX_DEST_RD_AT = 1u<<10, UAF_QP_RETRY = 1u<<11,
    UAF_QP_RNR_RETRY = 1u<<12, UAF_QP_PORT = 1u<<13,
    UAF_QP_TIMEOUT = 1u<<14,
};

struct uaf_qp_attr {
    uint32_t struct_size;
    uint32_t qp_state;
    uint32_t sq_psn;
    uint32_t rq_psn;
    uint32_t dest_qp_num;
    uint32_t path_mtu;      /* BYTES, including the 64-byte header */
    uint16_t dest_lid;
    uint8_t max_rd_atomic;
    uint8_t max_dest_rd_atomic;
    uint8_t retry_cnt;
    uint8_t rnr_retry;
    uint8_t port_num;
    uint8_t reserved0;
    uint8_t dest_gid[16]; /* Profile A */
    uint8_t dest_ip[16]; /* Profile B, IPv4-mapped permitted */
    uint16_t dest_udp_port;
    uint16_t src_udp_port;
    uint32_t rnr_timer_ns;
    uint64_t timeout_ns; /* 0 selects UAF_RTO_BASE_NS */
};
```

[R-4.5-001] uaf_create_qp() SHALL take a struct uaf_qp_init_attr. v2.0's create_qp took two completion queues and nothing else: no depth, no SGE limit, no queue-pair type, so

two implementations would allocate differently from the same call.

[R-4.5-002] `path_mtu` is expressed in **bytes and includes the 64-byte header**. v2.0 declared it `uint16_t` and its sample assigned 4096, while a verbs backend needs the IB enumeration (`IBV_MTU_4096 = 5`); the unit was never stated. A backend SHALL convert to its native encoding internally and SHALL reject a value that is not a supported path MTU.

[R-4.5-003] `uaf_modify_qp()` SHALL take an attribute mask. A transition SHALL fail with `UAF_ERR_INVALID` if a field it requires is absent from the mask. The required masks are in Section 10.1.

[R-4.5-004] `attr->qp_state` SHALL equal the state argument. A mismatch SHALL fail with `UAF_ERR_INVALID` rather than silently preferring one. v2.0's reference application set both and the specification never said which won.

4.6 Memory registration

[R-4.6-001] `uaf_reg_mr()` SHALL register the caller's memory **in place**. On success (`*mr`)->`addr == addr`. An implementation MUST NOT copy, bounce or relocate the buffer.

[R-4.6-002] A backend that cannot map arbitrary caller memory SHALL fail `uaf_reg_mr()` with `UAF_ERR_NOT_SUPPORTED` and SHALL clear `UAF_CAP_REG_ANY_MEM`.

[R-4.6-003] `uaf_alloc_mr()` SHALL allocate DMA-capable memory and register it in one step, returning the chosen address in (`*mr`)->`addr`. Every backend SHALL implement it. A portable application either tests `UAF_CAP_REG_ANY_MEM` or always allocates through `uaf_alloc_mr()`.

[R-4.6-004] `uaf_dereg_mr()` releases a region from `uaf_reg_mr()`; `uaf_free_mr()` releases one from `uaf_alloc_mr()`. Crossing them SHALL fail with `UAF_ERR_INVALID`.

Why this changed. v2.0's `sd_reg_mr()` allocated a second buffer, memcpy'd the caller's bytes into it, and reported the copy's address. Two consequences, neither documented: every later write the application made to its own buffer was invisible to the fabric, and "zero-copy by default" was false on that backend. The reference application in v2.0 Appendix B did exactly that and would have transmitted stale data.

4.7 Return conventions

[R-4.7-001] A **control** function returns `UAF_OK (0)` on success or a negative enum `uaf_error` on failure.

[R-4.7-002] A **poll** function — `uaf_poll_cq()` and `uaf_storage_poll()` — returns a non-negative count of entries written, from 0 to `max` inclusive, or a negative enum `uaf_error`. A return of 0 means the queue was empty and is not an error.

[R-4.7-003] A caller SHALL test `n < 0` for failure. Testing `n != UAF_OK` is incorrect and would treat a successful batch of one as an error.

[R-4.7-004] `UAF_ERR_CQ_EMPTY` is retired and its value (`-8`) reserved. No function returns it.

Why this changed. v2.0 stated as principle 4 that every API returns a typed error, while `poll_cq(cq, max, wc)` must also report how many completions it produced — and `UAF_OK` is 0, which is also "filled none". Its reference application looped on `== UAF_ERR_CQ_EMPTY`, a reading that forecloses the batch polling `max` exists for. The single most-used function on the data path had no defined contract.

4.8 Device capability query

```
struct uaf_device_attr {
    uint32_t struct_size;      /* caller sets to sizeof(...) */
    uint32_t wire_profiles;    /* enum uaf_wire_profile bitmask */
    uint64_t caps;             /* enum uaf_device_cap bitmask */
    uint32_t max_qp;           uint32_t max_cq;
    uint32_t max_mr;           uint32_t max_sge;
    uint32_t max_inline_data;  uint32_t max_cqe;
    uint32_t max_sqe;          uint32_t path_mtu_max;
    uint64_t max_msg_len;
    uint8_t max_rd_atomic;     uint8_t max_dest_rd_atomic;
    uint8_t reserved0[6];
};

int uaf_query_device(uaf_device_t *dev, struct uaf_device_attr *attr);
```

[R-4.8-001] `uaf_query_device()` is the only authoritative source of a device's limits and capabilities. v2.0 had no such call; an application could not discover max SGE, max inline, path MTU, atomic support, or even whether DST existed on the backend it was linked against.

[R-4.8-002] An application SHALL NOT use an OPTIONAL service whose capability bit is clear. Doing so SHALL fail with `UAF_ERR_NOT_SUPPORTED`.

4.9 Threading

[R-4.9-001] A queue pair, completion queue, storage queue and completion ring are **single-producer, single-consumer**. Concurrent `uaf_post_send()` on one queue pair from two threads, or concurrent `uaf_poll_cq()` on one completion queue, is undefined unless `UAF_CAP_MT_POST` is set.

[R-4.9-002] Distinct queue pairs MAY be used concurrently from distinct threads without external locking.

[R-4.9-003] `uaf_init()`, `uaf_fini()`, `uaf_query_device()` and the registration calls are thread-safe.

v2.0 had no threading section. Both of its ring samples were plainly single-producer — load the producer index, compute the next, store it, with no compare-and-swap — and nothing said so, so a correct-looking multi-threaded application would have silently corrupted a ring.

4.10 Backend loading

[R-4.10-001] Each backend library SHALL export exactly one symbol, `const struct uaf_pal_ops *uaf_pal_get_ops(void)`.

[R-4.10-002] The public headers MUST NOT declare per-backend symbols. v2.0's `uaf_pal.h` declared `uaf_pal_nvidia`, `uaf_pal_snapdragon` and `uaf_pal_dragonfly` together, so any consumer of the header had to link all three backends or take undefined symbols.

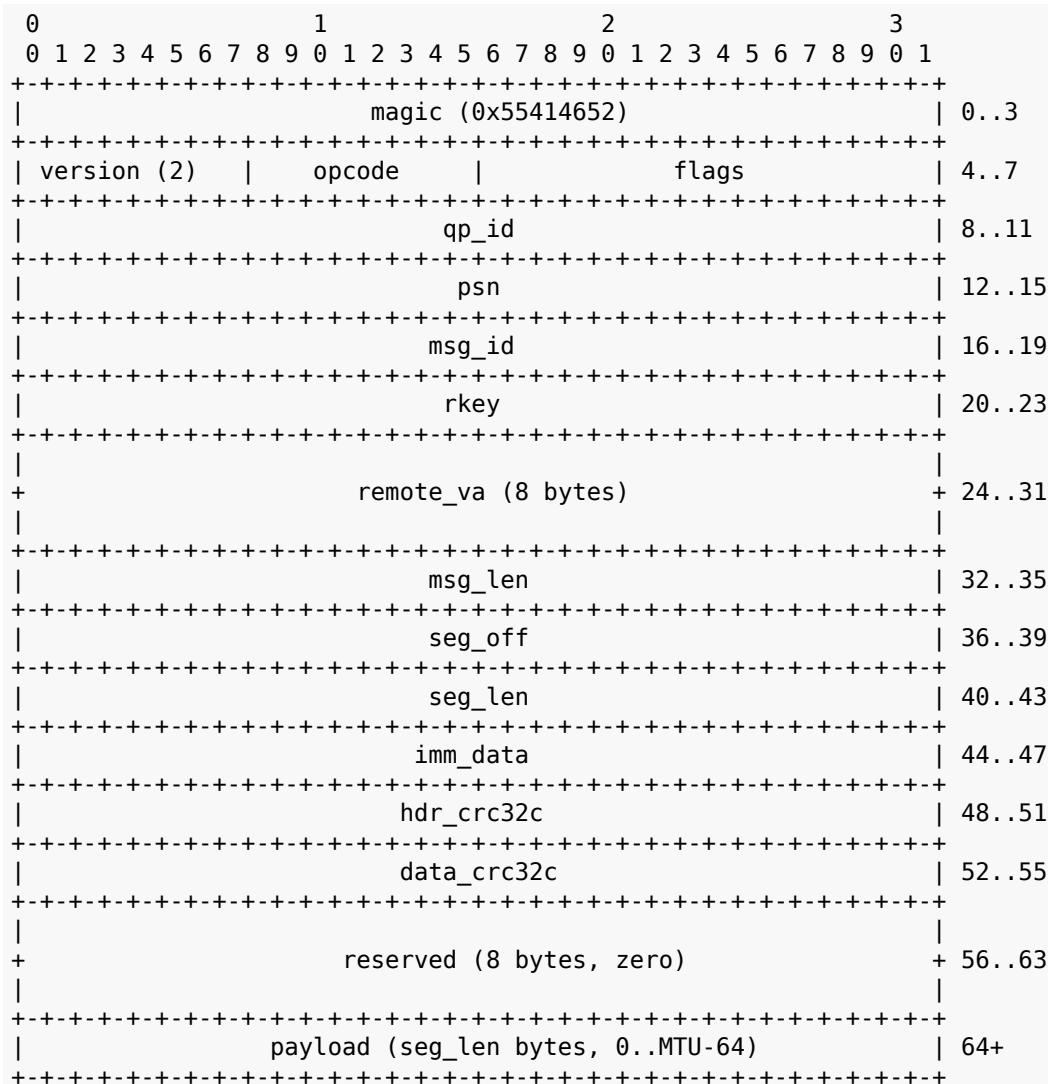
[R-4.10-003] `uaf_init()` SHALL resolve exactly one backend: the value of the `UAF_BACKEND` environment variable if set, otherwise the single backend library present. If none or more than one candidate is found, `uaf_init()` SHALL fail with `UAF_ERR_NODEV`.

5 RMT Protocol — Profile B (UAF-RMT over UDP)

[R-5.1-001] This section applies to UAF_PROFILE_UAFR only. A Profile A implementation follows the InfiniBand Architecture Specification for its wire behaviour and uses this document only for the API of Section 4.

5.1 Packet format

[R-5.1-002] All multi-byte wire fields SHALL be encoded in network byte order. The fixed header SHALL be 64 bytes.



5.2 Field definitions

Field	Offset	Size	Description
magic	0	4	0x55414652 ("UAFR")
version	4	1	Protocol version, 2

Field	Offset	Size	Description
opcode	5	1	enum uaf_wr_opcode or enum uaf_wire_opcode, per Section 5.3
flags	6	2	See below; bits 6..15 reserved and MUST be zero
qp_id	8	4	Destination queue-pair number
psn	12	4	Packet sequence number, incremented per packet
msg_id	16	4	Identifies the message a segment belongs to
rkey	20	4	32-bit remote key
remote_va	24	8	Base target address of the message , 8-byte aligned in the header
msg_len	32	4	Total length of the message in bytes
seg_off	36	4	Byte offset of this segment within the message
seg_len	40	4	Payload bytes carried by this packet
imm_data	44	4	Immediate data; also the expected PSN in a NAK
hdr_crc32c	48	4	CRC32C over header bytes 0..47
data_crc32c	52	4	CRC32C over the seg_len payload bytes; valid when UAF_WIRE_FLAG_DATA_CRC is set
reserved	56	8	MUST be zero on transmit and MUST be checked on receive

Flag	Bit	Meaning
UAF_WIRE_FLAG_INLINE	0	Payload is inline data, at most UAF_MAX_INLINE bytes
UAF_WIRE_FLAG_IMM	1	imm_data is meaningful
UAF_WIRE_FLAG_SOLICITED	2	Solicited event at the responder
UAF_WIRE_FLAG_ACK_REQ	3	Responder SHALL send an ACK
UAF_WIRE_FLAG_DATA_CRC	4	data_crc32c is valid
UAF_WIRE_FLAG_RETRY	5	This packet is a retransmission

[R-5.2-001] remote_va is the base address of the whole message. The target address of a segment is remote_va + seg_off. This keeps every segment self-describing without a second address field.

[R-5.2-002] A receiver SHALL reject a header whose reserved bytes or reserved flag bits are non-zero, with UAF_ERR_PROT0. The reserved area lies outside hdr_crc32c's coverage, so this check is explicit and not implied by the CRC.

v2.0's header spent 4 bytes on a reserved lkey beside 20 further reserved bytes, carried a single length with no message identity and no segment offset, and left the payload outside every integrity check.

5.3 Opcode semantics

[R-5.3-001] The following, and only the following, are wire opcodes.

Opcode	Value	Requires rkey	Payload
UAF_WR_SEND	0x01	No	0..MTU-64 bytes of the message
UAF_WR_SEND_INLINE	0x02	No	1..256 bytes; single segment only
UAF_WR_RDMA_READ	0x04	Yes	Request: none. msg_len is the length requested
UAF_WR_RDMA_WRITE	0x05	Yes	seg_len bytes written at remote_va + seg_off
UAF_WR_RDMA_WRITE_IMM	0x06	Yes	As WRITE; the last segment also delivers imm_data
UAF_WR_ATOMIC_CMP_SWP	0x07	Yes	16 bytes: compare, then swap
UAF_WR_ATOMIC_FETCH_ADD	0x08	Yes	8 bytes: addend
UAF_OP_RDMA_READ_RESP	0x84	No	seg_len bytes of the requested data
UAF_OP_ATOMIC_ACK	0x88	No	8 bytes: the pre-operation value
UAF_OP_CM_REQ / REP / RTU / REJ	0x80..0x83	No	Section 5.5
UAF_OP_ACK	0x90	No	None
UAF_OP_NAK	0x91	No	None; imm_data carries the expected PSN

[R-5.3-002] UAF_WR_RECV (0x03) SHALL NOT appear on the wire. It is a local receive-buffer post. v2.0's opcode table listed it as a packet with an empty payload, which gave implementers a packet to build that has no meaning. Conformance case C-3 rejects it at both encode and decode.

5.3.1 Framing

[R-5.3-003] A message of msg_len bytes SHALL be cut into segments of at most MTU - 64 bytes. Every segment of one message SHALL carry the same msg_id, msg_len and remote_va.

[R-5.3-004] FIRST and LAST SHALL be **derived**, never signalled:

$$\text{FIRST} \equiv (\text{seg_off} = 0), \quad \text{LAST} \equiv (\text{seg_off} + \text{seg_len} = \text{msg_len})$$

A sender therefore cannot contradict itself, and there is no flag for a receiver to disbelieve.

[R-5.3-005] A receiver SHALL raise **exactly one** work completion per message, on the LAST segment, with `byte_len = msg_len`. If the bytes accepted do not total `msg_len`, the message SHALL be completed with `UAF_ERR_PROTO`.

[R-5.3-006] A zero-length message occupies exactly one segment with `msg_len = seg_len = 0`.

[R-5.3-007] `seg_off + seg_len` SHALL NOT exceed `msg_len`. A violation is `UAF_ERR_PROTO` on receive and `UAF_ERR_INVALID` on send.

v2.0 could not express a message larger than one packet. Every packet carried `remote_va` and `length` with no message identity, no total length, no segment offset and no first/middle/last indication, so a 1 MiB RDMA WRITE had no encoding and a responder had no way to know when to raise one completion.

5.4 Atomics

[R-5.4-001] Atomic operands SHALL be 8 bytes, big-endian.

Packet	Opcode	seg_len	Payload
Compare-and-swap request	0x07	16	compare (8) then swap (8)
Fetch-and-add request	0x08	8	addend (8)
Atomic acknowledgement	0x88	8	value before the operation (8)

[R-5.4-002] `remote_va` for an atomic SHALL be 8-byte aligned. A responder receiving a mis-aligned atomic SHALL NOT perform the operation and SHALL reply `UAF_OP_NAK`; the local API rejects it with `UAF_ERR_INVALID`.

[R-5.4-003] The acknowledgement SHALL echo the request's `msg_id`.

[R-5.4-004] The requester SHALL write the returned value into SGE 0 of the originating work request, which SHALL be at least 8 bytes and locally writable, and SHALL set `UAF_WC_ATOMIC_ORIG` with `byte_len = 8`. A work request whose SGE 0 is smaller than 8 bytes SHALL fail with `UAF_ERR_INVALID`.

[R-5.4-005] Atomics are atomic with respect to other UAF atomics on the same responder, and with respect to RDMA READ and WRITE targeting the same 8 bytes.

[R-5.4-006] Atomicity with respect to **host CPU** accesses on UAF-D REQUIRES CXL.cache and therefore `UAF_ENABLE_CXL_CACHE`. A build without it SHALL clear `UAF_CAP_ATOMICS`.

[R-5.4-007] `max_dest_rd_atomic` bounds the outstanding atomic and READ requests a responder will accept. v2.0 had `max_rd_atomic` and no responder-side limit at all.

v2.0 specified a 16-byte request payload and never defined the response. The fetched value had nowhere to go: `uaf_wc` had no field for it and no local SGE semantics were stated. Atomics were unimplementable.

5.5 Connection management

[R-5.5-001] A CM packet SHALL consist of the 64-byte header of Section 5.1 with a CM opcode, followed by a 64-byte connection record, followed by a 32-byte authenticator. `seg_len` SHALL be 96 and `msg_len` SHALL equal `seg_len`.

5.5.1 Connection record (64 bytes, big-endian)

Field	Offset	Size	Description
qp_num	0	4	Sender's queue-pair number
psn	4	4	Sender's initial PSN
rkey	8	4	Remote key for the region being offered
lid	12	2	Profile A only; zero under Profile B
mtu	14	2	Path MTU in bytes, including the header
remote_va	16	8	Base of the offered region
cxl_base	24	8	UAF-D CXL window offset
cxl_size	32	8	UAF-D CXL window length
gid	40	16	Profile A GID; zero under Profile B
sl	56	1	Service level
traffic_class	57	1	Traffic class
wire_profile	58	1	Exactly one enum uaf_wire_profile bit
reserved	59	5	MUST be zero

[R-5.5-002] struct uaf_conn_info is a host-local struct with interior padding and host byte order. It SHALL NOT be written to a socket. An implementation SHALL use uaf_conn_info_serialize() and uaf_conn_info_deserialize(). v2.0's uaf_conn_info had 6 bytes of interior padding, no byte-order rule, and was evidently meant to be memcpy'd to a peer.

[R-5.5-003] wire_profile SHALL name exactly one profile. Zero, or more than one bit, is UAF_ERR_PROTO.

[R-5.5-004] mtu SHALL exceed 64. A smaller value leaves no room for the header and is UAF_ERR_PROTO.

[R-5.5-005] A connection record SHALL NOT contain a file descriptor. A descriptor is a process-local integer: copied to another node it is meaningless, and interpreted locally it is a confused-deputy bug. v2.0 carried dma_buf_fd in uaf_conn_info. Descriptors now appear only in struct uaf_peer_info, which is same-host only and whose descriptor SHALL be transferred out of band with SCM_RIGHTS over an AF_UNIX socket.

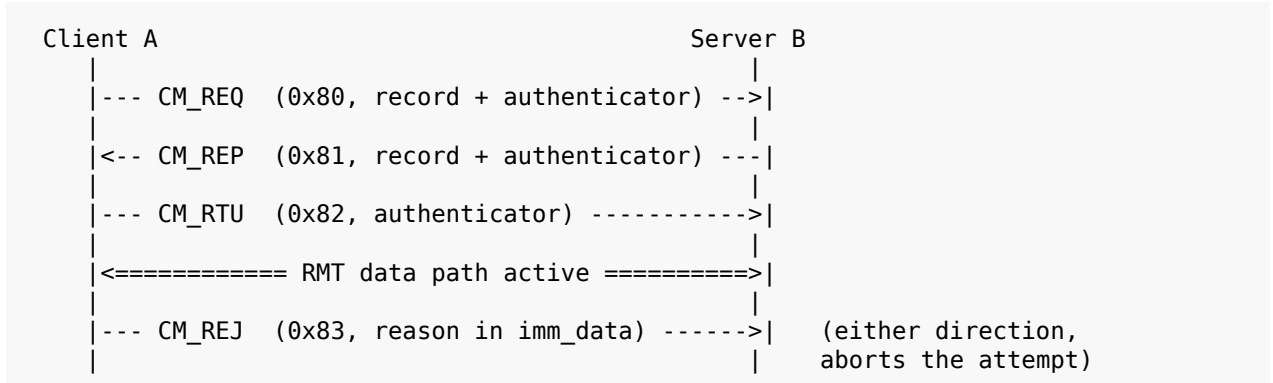
5.5.2 Authenticator (32 bytes)

Field	Offset	Size	Description
nonce	0	8	Sender's random nonce, from a CSPRNG
timestamp_ns	8	8	Sender's clock, nanoseconds
mac	16	16	HMAC-SHA256, truncated to 128 bits

[R-5.5-006] The MAC SHALL be computed over the concatenation of header bytes 0..47, the 64-byte connection record, and the nonce and timestamp, keyed by a pre-shared fabric key. A packet failing the MAC check SHALL be dropped with UAF_ERR_AUTH and SHALL NOT alter queue-pair state.

[R-5.5-007] A responder SHALL reject a CM packet whose timestamp_ns is outside UAF_CM_REPLAY_WINDOW_NS of its own clock, or whose (nonce, qp_num) pair it has already accepted inside that window.

5.5.3 Handshake



[R-5.5-008] The passive side SHALL move to RTR on accepting CM_REQ and to RTS on accepting CM_RTU. The active side SHALL move to RTR on accepting CM_REP and to RTS before sending CM_RTU.

[R-5.5-009] **Simultaneous open.** If both sides send CM_REQ before either receives one, the side with the numerically lower (gid, qp_num) under Profile A, or lower (dest_ip, dest_udp_port, qp_num) under Profile B, SHALL continue as the active side and the other SHALL abandon its own CM_REQ and answer with CM_REP.

[R-5.5-010] **Teardown.** uaf_disconnect() SHALL send CM_REJ with reason UAF_CM_REASON_CLOSE, flush outstanding work with UAF_ERR_QP_STATE, and move the queue pair to RESET. A CM_REJ received on an established connection SHALL be treated identically.

[R-5.5-011] Retransmission of CM_REQ, CM_REP and CM_RTU SHALL use the timers of Section 5.6. An unanswered CM_REQ after UAF_RETRY_MAX attempts SHALL fail uaf_connect() with UAF_ERR_TIMEOUT.

v2.0's Section 5.4 was a three-arrow diagram. There was no packet layout, no address format, no authentication, no simultaneous-open rule, no teardown, no reject, and the opcodes it named appeared in no enum. uaf_connect() existed in the header and the reference application never called it.

5.6 Reliability

[R-5.6-001] Profile B SHALL use Go-Back-N with cumulative acknowledgement. psn increments once per packet and wraps modulo 2^{32} .

[R-5.6-002] A sender that may be bridged to Profile A SHALL keep psn within 24 bits, because the IB transport carries a 24-bit PSN.

[R-5.6-003] A receiver SHALL drop a packet whose psn is not the expected one and SHOULD reply UAF_OP_NAK with the expected PSN in imm_data.

[R-5.6-004] The send window SHALL be at least the bandwidth-delay product of the path, in segments:

$$W_{\min} = \left\lceil \frac{R \cdot \text{RTT}}{8 \cdot L_{\text{seg}}} \right\rceil$$

for line rate R bit/s, round trip RTT seconds and segment payload L_{seg} bytes. The default UAF_WINDOW_DEFAULT is 256 segments, which covers 800 Gbps at a 10 μ s round trip with 4032-byte segments ($W_{\min} = 248$).

[R-5.6-005] The retransmit timeout SHALL start at UAF_RTO_BASE_NS (50 μ s), double on each expiry, and saturate at UAF_RTO_MAX_NS (2 ms). After UAF_RETRY_MAX expires the queue pair SHALL move to UAF_QPS_ERR and the outstanding work SHALL complete with UAF_ERR_TIMEOUT.

[R-5.6-006] rnr_retry and rnr_timer_ns govern the responder-not-ready case: a responder with no posted receive buffer SHALL reply NAK-RNR, and the requester SHALL wait rnr_timer_ns and retry up to rnr_retry times before failing with UAF_ERR_TIMEOUT. v2.0 declared rnr_retry and gave it no behaviour.

[R-5.6-007] A Profile A implementation SHALL NOT run these timers. IB RC retransmission is performed by the adapter, and a second software timer over it would produce duplicate retransmission. v2.0 specified one software timer for all backends while also claiming hardware offload on UAF-N.

v2.0 specified a 100 ms base timeout in a fabric whose stated target is 1.5 μ s — roughly 6×10^4 round trips — and a fixed 64-segment window, which caps a 400 Gbps link at about 205 Gbps once the round trip reaches 10 μ s.

5.7 Error handling

Condition	Action	Reported as
magic mismatch	Drop silently; queue-pair state unchanged	—
version unknown	Drop silently; state unchanged	—
hdr_crc32c mismatch	Drop; count; state unchanged	UAF_ERR_CRC
data_crc32c mismatch	Drop; do not write the payload; NAK	UAF_ERR_CRC
Reserved bytes or flags non-zero	Drop	UAF_ERR_PROTO
Unexpected psn	Drop; NAK with expected PSN	—
Unknown or unbound rkey	Drop; count against UAF_RKEY_FAIL_MAX	UAF_ERR_RKEY
Access outside the region	Drop; complete the request	UAF_ERR_MR_FAULT
Access flags insufficient	Drop; complete the request	UAF_ERR_PERM
CM MAC or replay check failed	Drop	UAF_ERR_AUTH
Atomic target misaligned	NAK; do not perform the operation	UAF_ERR_INVALID

[R-5.7-001] A malformed or unauthenticated packet MUST NOT move a queue pair to UAF_QPS_ERR. Otherwise any host able to reach the UDP port could terminate a connection with one datagram.

5.8 Test vectors

[R-5.8-001] An implementation SHALL reproduce the following values. They are produced and checked by conformance case C-2.

CRC32C is Castagnoli, reflected, polynomial 0x1EDC6F41 (reflected 0x82F63B78), initial value 0xFFFFFFFF, final XOR 0xFFFFFFFF.

Input	CRC32C
32 bytes of 0x00	0x8A9136AA
32 bytes of 0xFF	0x62A8AB43
32 bytes, 0x00 through 0x1F	0x46DD794E

Input	CRC32C
"123456789"	0xE3069283
"a"	0xC1D04330
empty	0x00000000

Golden header. An RDMA WRITE, opcode 0x05, flags = ACK_REQ | DATA_CRC (0x0018), qp_id 0x00000101, psn 1, msg_id 7, rkey 0xDEADBEEF, remote_va 0x0000100000000000, msg_len 16, seg_off 0, seg_len 16, imm_data 0, payload the 16 ASCII bytes UAF-SPEC-001 v21:

```
00: 55 41 46 52 02 05 00 18 00 00 01 01 00 00 00 01
16: 00 00 00 07 DE AD BE EF 00 00 10 00 00 00 00 00
32: 00 00 00 10 00 00 00 00 00 00 00 10 00 00 00 00
48: CD 6B 9B 1A 89 50 EC 50 00 00 00 00 00 00 00 00
```

hdr_crc32c = 0xCD6B9B1A over bytes 0..47. data_crc32c = 0x8950EC50 over the 16 payload bytes.

Connection record. qp_num 0x01020304, psn 0x00ABCDEF, rkey 0xCAFEBAFE, lid 0x1234, mtu 4096, remote_va 0x00001000000002000, cxl_base 0x0000200000000000, cxl_size 0x0000000040000000, gid 0xE0..0xEF, sl 3, traffic_class 7, wire_profile 2 serialises to 64 bytes whose mtu field reads 10 00 and whose final 5 bytes are zero; conformance case C-9 checks the round trip and that a host-struct memcpy does *not* produce the same bytes.

5.9 Congestion control

[R-5.9-001] A Profile A implementation SHALL use the adapter's DCQCN and SHALL declare UAF_CAP_HW_CC.

[R-5.9-002] A Profile B implementation SHALL implement the following rate controller. v2.0 gave two equations with α never defined or updated, no time constant, no R_{AI} , no minimum rate, and no statement of whether a mark is an IP ECN codepoint or a RoCE CNP — which is not enough for two implementations to interoperate without one starving the other.

Congestion is signalled by the IP ECN codepoint CE on a received data packet, which the receiver reflects in the next ACK. Let F_k be the fraction of acknowledged bytes marked in interval k , g the smoothing weight, T the update period, R_{AI} the additive-increase step and $R_{\min}$ the floor:

$$\alpha_k = (1 - g) \alpha_{k-1} + g F_k$$

$$R_{k+1} = \max\left(R_{\min}, R_k \left(1 - \frac{\alpha_k}{2}\right)\right) \quad \text{if } F_k > 0$$

$$R_{k+1} = \min(R_{\text{line}}, R_k + R_{AI}) \quad \text{if } F_k = 0$$

[R-5.9-003] Defaults SHALL be $g = 1/16$, $T = 50 \mu\text{s}$, $R_{AI} = R_{\text{line}}/256$, $R_{\min} = R_{\text{line}}/1024$.

[R-5.9-004] The rate SHALL be reduced at most once per update period T . Without this bound a burst of marks collapses the rate geometrically toward the floor.

6 DST Protocol

6.1 Structure of this section

v2.0 Section 6.1 specified a 64-byte “UAF SQE” with its own crc32c field. No backend filled it; every sample built an NVMe submission entry instead, and the coverage of that CRC was never stated. The pseudo-format is **deleted**. DST is specified here as a mapping from struct uaf_storage_cmd onto the NVMe I/O command set, which is what implementations actually emit.

[R-6.1-001] A DST implementation SHALL submit NVMe I/O commands as defined by NVM Express Base 2.0d. It SHALL NOT define an intermediate wire format.

[R-6.1-002] Queue creation, the admin queue, Identify Controller, Identify Namespace, namespace formatting and completion-queue association are preconditions and are outside this specification. uaf_storage_open() SHALL fail with UAF_ERR_NODEV if they are not satisfied. v2.0 assumed all of them silently.

6.2 Command mapping

```
struct uaf_storage_cmd {
    uint32_t struct_size;
    uint32_t opcode;           /* enum uaf_wr_opcode, STORAGE_* only */
    uint64_t wr_id;
    uint64_t lba;
    uint32_t nlb;             /* 1-BASED block count; 0 is invalid */
    uint32_t buf_offset;
    void *buf;
    uint64_t buf_len;         /* was size_t in v2.0: not ABI-stable */
    uint32_t flags;
    uint32_t reserved0;
};
```

[R-6.2-001] The opcode map SHALL be total. Every value not in the table is UAF_ERR_INVALID.

uaf_storage_cmd.opcode	NVMe opcode
UAF_WR_STORAGE_READ (0x10)	0x02 Read
UAF_WR_STORAGE_WRITE (0x11)	0x01 Write
UAF_WR_STORAGE_FLUSH (0x12)	0x00 Flush

v2.0’s sample selected the opcode with a two-way ternary on READ, so UAF_WR_STORAGE_FLUSH fell through the WRITE arm and a flush was issued as a write. Conformance case C-8 asserts the flush mapping explicitly and asserts that it is not the write opcode.

[R-6.2-002] nlb is a **1-based** block count in the API. NVMe CDW12 carries a **0-based** count. The library performs the conversion, SHALL reject nlb == 0 with UAF_ERR_INVALID, and SHALL reject nlb > 65536.

nlb (API)	CDW12 NLB	Result
0	—	UAF_ERR_INVALID
1	0x0000	One block
8	0x0007	Eight blocks
65536	0xFFFF	Largest legal transfer
65537 or more	—	UAF_ERR_INVALID

v2.0 computed $(\text{cmd} \rightarrow \text{n} \text{lb} - 1) \& 0\text{xFFF}$ with no zero check, so a zero-length request became a 65536-block transfer — 256 MiB at a 4 KiB block size. Neither the API nor the diagram said whether the on-ring value was 0-based or 1-based.

[R-6.2-003] The data pointer SHALL be selected from the buffer’s IOVA and length against the controller’s memory page size. A single PRP entry describes at most one page-boundary crossing.

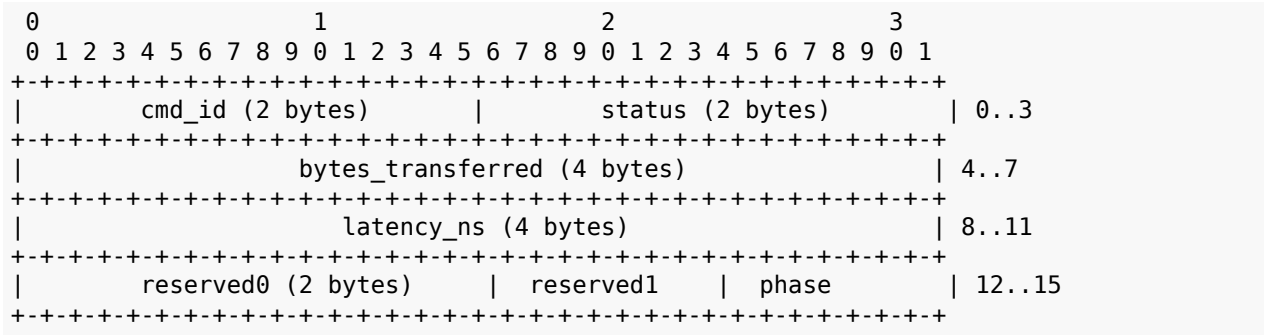
Condition	Data pointer
Buffer ends within the first page	PRP1 only
Buffer crosses exactly one page boundary	PRP1 and PRP2
More than two pages, controller without SGL support	PRP2 points at a PRP list
More than two pages, controller with SGL support	NVMe SGL

v2.0 assigned the whole buffer to `dp1r_prp1` regardless of length, which is correct only for a buffer that fits in one page.

[R-6.2-004] `buf_len` SHALL be at least `nlb` multiplied by the namespace block size, after `buf_offset`. A shorter buffer is `UAF_ERR_INVALID`.

6.3 Completion ring

[R-6.3-001] A completion ring entry SHALL be exactly 16 bytes, naturally aligned, and SHALL NOT be packed.



Field	Offset	Size	Description
<code>cmd_id</code>	0	2	Command identifier allocated by the library
<code>status</code>	2	2	<code>int16_t</code> , an enum <code>uaf_error</code> value
<code>bytes_transferred</code>	4	4	Bytes actually moved
<code>latency_ns</code>	8	4	Clamped at <code>UINT32_MAX</code> , never wrapped
<code>reserved0</code>	12	2	MUST be zero
<code>reserved1</code>	14	1	MUST be zero
<code>phase</code>	15	1	Bit 0 is the phase tag; bits 7..1 reserved

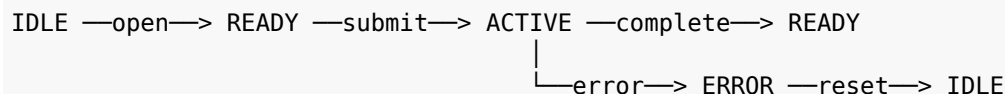
[R-6.3-002] `phase` SHALL be the **last byte** of the entry and SHALL be the only byte a consumer reads before the remainder is known valid. A producer SHALL write bytes 0..14, issue a release barrier, and then write `phase`.

[R-6.3-003] A ring SHALL be zero-filled at creation and the initial expected phase SHALL be 1. A consumer SHALL read `phase`, compare it with the expected phase, and only then read the rest of the entry. The expected phase SHALL flip on every wrap.

[R-6.3-004] A ring base SHALL be UAF_CQE_ALIGN (16-byte) aligned, and the depth SHALL be a power of two.

Why this matters. v2.0's CQE had 4 reserved bytes and no phase tag, and its status field encoded success as UAF_OK = 0 — the same bits a zeroed slot holds. A poller therefore could not distinguish a fresh completion from an untouched one, and DST completion polling was unimplementable. v2.0 also marked the struct packed, which drops its alignment to 1, defeats the aligned 16-byte load a ring wants, and is undefined behaviour on strict-alignment targets. Conformance case C-7 polls a zeroed ring and requires 0 completions.

6.4 Storage queue state machine



[R-6.4-001] `uaf_storage_reset(sq, cr)` drives the ERROR to IDLE edge. v2.0 drew that edge with no entry point able to traverse it.

[R-6.4-002] `uaf_storage_close(sq, cr)` SHALL release **both** objects that `uaf_storage_open()` created. v2.0's close took only the submission queue, so the completion ring — and on UAF-N the cuFile batch handle it shares — was never released.

6.5 Command identifiers

[R-6.5-001] `cmd_id` SHALL be allocated by the library as an index into an in-flight table, and the library SHALL restore the caller's full 64-bit `wr_id` before delivering a completion.

[R-6.5-002] A `cmd_id` SHALL NOT be reused while its command is in flight. When the table is full, `uaf_storage_submit()` SHALL return UAF_ERR_BUSY.

[R-6.5-003] Releasing an unknown or already-free `cmd_id` is UAF_ERR_PROTO.

v2.0 defined `cmd_id` as the low 16 bits of `wr_id`. Two in-flight commands whose `wr_ids` differed only above bit 16 produced the same `cmd_id`, and since the completion carried nothing else, the application could not recover its own identifier. RMT delivered a full 64-bit `wr_id` in `uaf_wc`; DST silently narrowed it. Conformance case C-8 submits `0x0000000000001234` and `0xAAAABBBB00001234` together and requires both to come back intact.

6.6 Submission ring memory type

[R-6.6-001] A submission ring a device reads SHALL be either device-coherent, or explicitly cleaned to memory before the doorbell is written. A cacheable ring plus a bare doorbell is a stale-descriptor bug.

[R-6.6-002] Doorbell ordering follows Section 9.4.

7 Platform Backend: UAF-N (NVIDIA)

Listings in this section are INFORMATIVE ([R-1.4-003]).

7.1 Profile and capabilities

[R-7.1-001] UAF-N SHALL declare UAF_PROFILE_IBV, UAF_CAP_HW_CC, and UAF_CAP_REG_ANY_MEM. It SHALL declare UAF_CAP_GPU_MR only if it implements the dma-buf registration path of [R-7.3-001].

7.2 Device discovery

[R-7.2-001] An `ibv_device` pointer obtained from `ibv_get_device_list()` SHALL NOT be used after `ibv_free_device_list()`. An implementation SHALL either open the device before freeing the list, or retain the list for the lifetime of its device array.

```
struct uaf_n_devs {
    struct ibv_device **list; /* retained: freeing it invalidates entries */
    int count;
};

static int nv_get_device_list(uaf_device_t ***devs, int *count)
{
    struct uaf_n_devs *s = calloc(1, sizeof(*s));
    if (!s) return UAF_ERR_NOMEM;

    s->list = ibv_get_device_list(&s->count);
    if (!s->list || s->count <= 0) { free(s); return UAF_ERR_NODEV; }

    uaf_device_t **out = calloc((size_t)s->count, sizeof(*out));
    if (!out) { ibv_free_device_list(s->list); free(s); return UAF_ERR_NOMEM; }

    for (int i = 0; i < s->count; i++) {
        struct uaf_n_dev *d = calloc(1, sizeof(*d));
        if (!d) { nv_unwind(out, i, s); return UAF_ERR_NOMEM; }
        d->ibv_dev = s->list[i]; /* valid while s->list is retained */
        d->owner = s;
        d->name = strdup(ibv_get_device_name(s->list[i]));
        if (!d->name) { free(d); nv_unwind(out, i, s); return UAF_ERR_NOMEM; }
        out[i] = &d->pub;
    }
    *devs = out; *count = s->count;
    return UAF_OK; /* s->list freed in free_device_list */
}
```

v2.0 stored `list[i]` into its device array and then called `ibv_free_device_list(list)`, leaving every retained pointer dangling. It also left `strdup` and each per-device `calloc` unchecked.

7.3 Memory registration

[R-7.3-001] Registering GPU memory SHALL use `ibv_reg_dma_buf_mr()` with a descriptor exported from CUDA, or the peer-memory client where the platform provides one. Plain `ibv_reg_mr()` on a CUDA device pointer is not the GPUDirect RDMA path.

[R-7.3-002] UAF_MR_GPU SHALL select that path. v2.0 defined the flag and used it nowhere, so it had no behaviour at all; UAF_MR_CXL was likewise unused.

[R-7.3-003] A registration failure SHALL be mapped from `errno`, not reported uniformly as UAF_ERR_NOMEM:

errno	UAF error
ENOMEM	UAF_ERR_NOMEM
EINVAL	UAF_ERR_INVALID
EACCES, EPERM	UAF_ERR_PERM
EFAULT	UAF_ERR_MR_FAULT
EOPNOTSUPP, ENOTSUP	UAF_ERR_NOT_SUPPORTED
anything else	UAF_ERR_NODEV

```
static int nv_reg_mr(uaf_domain_t *pd, void *addr, size_t len,
                   uint64_t flags, uaf_mr_t **out)
{
    int access = 0;
    if (flags & UAF_MR_LOCAL_WRITE) access |= IBV_ACCESS_LOCAL_WRITE;
    if (flags & UAF_MR_REMOTE_WRITE) access |= IBV_ACCESS_REMOTE_WRITE;
    if (flags & UAF_MR_REMOTE_READ) access |= IBV_ACCESS_REMOTE_READ;
    if (flags & UAF_MR_ATOMIC) access |= IBV_ACCESS_REMOTE_ATOMIC;
    access |= IBV_ACCESS_RELAXED_ORDERING; /* required for GPU peer DMA */

    struct uaf_n_mr *m = calloc(1, sizeof(*m));
    if (!m) return UAF_ERR_NOMEM;

    struct ibv_mr *ib;
    if (flags & UAF_MR_GPU) {
        int dmabuf_fd = -1;
        int rc = nv_export_cuda_dmabuf(addr, len, &dmabuf_fd); /* [R-7.3-001] */
        if (rc != UAF_OK) { free(m); return rc; }
        ib = ibv_reg_dmabuf_mr(pd->ibv_pd, 0, len,
                              (uint64_t)(uintptr_t)addr, dmabuf_fd, access);
        if (ib) m->dmabuf_fd = dmabuf_fd; else close(dmabuf_fd);
    } else {
        ib = ibv_reg_mr(pd->ibv_pd, addr, len, access);
    }
    if (!ib) { int rc = nv_errno_to_uaf(errno); free(m); return rc; }

    m->ibv_mr = ib; /* private; not in the public MR */
    m->pub.struct_size = (uint32_t)sizeof(uaf_mr_t);
    m->pub.pd = pd;
    m->pub.addr = addr; /* in place: [R-4.6-001] */
    m->pub.length = len;
    m->pub.lkey = ib->lkey;
    m->pub.rkey = ib->rkey;
    m->pub.flags = flags;
    *out = &m->pub;
    return UAF_OK;
}
```

7.4 DST via cuFile

[R-7.4-001] A UAF-N DST implementation SHALL call `cuFileDriverOpen()` once, SHALL register every data buffer with `cuFileBufRegister()`, SHALL submit with `cuFileBatchIOSubmit()`, and SHALL reap with `cuFileBatchIOGetStatus()`.

[R-7.4-002] The doorbell mechanism is internal to cuFile and the NVMe driver. This specification SHALL NOT describe it as a UAF-visible operation. v2.0's doorbell table claimed "cuFileBatchIOSubmit batches NVMe SQ tail updates via `nvidia-fs/BAR0`", which describes a kernel path rather than the API being specified.

```

static int nv_storage_open(uaf_domain_t *pd, const char *path,
                           uaf_sq_t **out_sq, uaf_cr_t **out_cr)
{
    if (nv_driver_open_once() != UAF_OK) return UAF_ERR_NODEV;

    int fd = open(path, O_RDWR | O_DIRECT | O_CLOEXEC);
    if (fd < 0) return nv_errno_to_uaf(errno);

    CUfileDescr_t desc = {0};
    desc.type          = CU_FILE_HANDLE_TYPE_OPAQUE_FD;
    desc.handle.fd     = fd;

    CUfileHandle_t fh;
    if (cuFileHandleRegister(&fh, &desc).err != CU_FILE_SUCCESS) {
        close(fd); return UAF_ERR_NODEV;
    }
    CUfileBatchHandle_t batch;
    if (cuFileBatchIOSetUp(&batch, UAF_SQ_DEPTH_DEFAULT).err
        != CU_FILE_SUCCESS) {
        cuFileHandleDeregister(fh); close(fd); return UAF_ERR_NOMEM;
    }

    struct uaf_n_sq *sq = calloc(1, sizeof(*sq));
    struct uaf_n_cr *cr = calloc(1, sizeof(*cr));
    if (!sq || !cr) {
        free(sq); free(cr); cuFileBatchIODestroy(batch);
        cuFileHandleDeregister(fh); close(fd); return UAF_ERR_NOMEM;
    }
    if (uaf_cid_init(&sq->cids, UAF_SQ_DEPTH_DEFAULT) != UAF_OK) {
        free(sq); free(cr); cuFileBatchIODestroy(batch);
        cuFileHandleDeregister(fh); close(fd); return UAF_ERR_NOMEM;
    }
    sq->fh = fh; sq->batch = batch; sq->fd = fd;
    cr->batch = batch; cr->owner = sq; /* closed together: [R-6.4-002] */
    *out_sq = &sq->pub; *out_cr = &cr->pub;
    return UAF_OK;
}

```

8 Platform Backend: UAF-S (Snapdragon, Linux)

Listings in this section are INFORMATIVE ([R-1.4-003]).

[R-8.1-001] UAF-S SHALL declare UAF_PROFILE_UAFR. It SHALL NOT declare UAF_CAP_HW_CC: its congestion control is the software controller of Section 5.9.

[R-8.1-002] This backend is Linux-only. DirectStorage is a Windows C++ API and is out of scope ([R-1.2-001]); v2.0 listed it in the same backend row as Linux dma-heap and FastRPC.

8.1 Memory registration

[R-8.2-001] sd_reg_mr() SHALL map the caller's pages for device access in place and SHALL NOT allocate a second buffer. If the SMMU mapping of caller memory is unavailable, it SHALL return UAF_ERR_NOT_SUPPORTED and the backend SHALL clear UAF_CAP_REG_ANY_MEM.

[R-8.2-002] `sd_alloc_mr()` SHALL allocate from `/dev/dma_heap/system`, map it, and register it. This is the portable path.

[R-8.2-003] A non-coherent buffer SHALL be synchronised with `DMA_BUF_IOCTL_SYNC` around every CPU access window. A barrier does not flush a dirty cache line.

[R-8.2-004] A length passed to the FastRPC layer SHALL be range-checked before narrowing. v2.0 passed `(int)len` from a `size_t`.

[R-8.2-005] Keys SHALL come from the CSPRNG key table of Section 11.2. A descriptor number SHALL NOT be used as a key, and a key SHALL NOT be derived from one.

```
struct uaf_s_mr { uaf_mr_t pub; int dma_buf_fd; int owns_mmap; };

static int sd_alloc_mr(uaf_domain_t *pd, size_t len, uint64_t flags,
                      uaf_mr_t **out)
{
    if (len == 0u || len > (size_t)INT_MAX) return UAF_ERR_INVAL;

    int heap_fd = open("/dev/dma_heap/system", O_RDWR | O_CLOEXEC);
    if (heap_fd < 0) return UAF_ERR_NODEV;

    struct dma_heap_allocation_data alloc = {
        .len = len, .fd_flags = O_RDWR | O_CLOEXEC,
    };
    if (ioctl(heap_fd, DMA_HEAP_IOCTL_ALLOC, &alloc) < 0) {
        close(heap_fd); return UAF_ERR_NOMEM;
    }
    close(heap_fd);

    void *va = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_SHARED,
                    alloc.fd, 0);
    if (va == MAP_FAILED) { close(alloc.fd); return UAF_ERR_NOMEM; }

    /* Map into the Hexagon SMMU context. Return value is checked. */
    int rc = remote_register_buf_attr(va, (int)len, alloc.fd,
                                      FASTRPC_ATTR_NON_COHERENT);
    if (rc != 0) { munmap(va, len); close(alloc.fd); return UAF_ERR_PERM; }

    struct uaf_s_mr *m = calloc(1, sizeof(*m));
    if (!m) { munmap(va, len); close(alloc.fd); return UAF_ERR_NOMEM; }

    uint32_t lkey = 0, rkey = 0;
    rc = uaf_key_register(&pd->keys, (uint64_t)(uintptr_t)va, len, flags,
                        0u, &lkey, &rkey); /* [R-8.2-005] */
    if (rc != UAF_OK) {
        free(m); munmap(va, len); close(alloc.fd); return rc;
    }

    m->dma_buf_fd = alloc.fd;
    m->owns_mmap = 1;
    m->pub.struct_size = (uint32_t)sizeof(uaf_mr_t);
    m->pub.pd = pd;
    m->pub.addr = va; /* the allocator chose it: [R-4.6-003] */
    m->pub.length = len;
    m->pub.lkey = lkey;
    m->pub.rkey = rkey;
    m->pub.flags = flags;
    *out = &m->pub;
    return UAF_OK;
}
```

```
}
```

v2.0's `sd_reg_mr()` allocated a dma-heap buffer, `memcpy`'d the caller's bytes into it and returned the copy's address as `mr->addr`, so later writes to the caller's own buffer never reached the fabric. It set `lkey = dma_buf_fd` and `rkey = fd ^ 0xA5A5A5A5`, a reversible function of a small, reused integer. It ignored `remote_register_buf_attr()`'s return value and left two `calloc` results unchecked.

8.2 Send path

[R-8.3-001] A send SHALL take exactly one path. `sd_post_send()` SHALL either enqueue to the hardware ring **or** transmit over the socket, never both. v2.0 enqueued the work request and then also called `sd_send_via_socket()`, so a request could be submitted twice.

[R-8.3-002] A socket transmission SHALL serialise the work request into the Profile B packet of Section 5.1. A queued struct `uaf_wr` contains pointers and is not a packet.

[R-8.3-003] Doorbell ordering follows Section 9.4. The doorbell value SHALL be the new ring tail, not the queue-pair number: a device cannot infer a tail from an identifier.

```
static int sd_post_send(uaf_qp_t *qp, struct uaf_wr *wr)
{
    struct uaf_s_qp *q = uaf_s_qp(qp);
    int rc = sd_validate_wr(q, wr);           /* num_sge, inline, opcode */
    if (rc != UAF_OK) return rc;

    if (q->mode == UAF_S_MODE_SOCKET)        /* [R-8.3-001] one path */
        return sd_send_serialized(q, wr);

    uint32_t slot;
    struct uaf_wr *e = uaf_ring_produce(&q->sq, &slot);
    if (!e) return UAF_ERR_BUSY;

    *e = *wr;                               /* [R-4.4-002] copy the WR */
    memcpy(q->sge_pool + slot * q->max_sge, wr->sge,
           wr->num_sge * sizeof(struct uaf_sge)); /* and the SGE array */
    e->sge = q->sge_pool + slot * q->max_sge;
    uaf_ring_commit(&q->sq);

    sd_dma_sync_for_device(q);              /* [R-8.2-003] */
    uaf_mmio_write32(q->doorbell, q->sq.tail); /* [R-8.3-003], [R-9.4-001] */
    return UAF_OK;
}
```

9 Platform Backend: UAF-D (Dragonfly)

Listings in this section are INFORMATIVE ([R-1.4-003]).

[R-9.1-001] UAF-D SHALL declare `UAF_PROFILE_UAFR` for node-to-node traffic. Its intra-host path is CXL.mem load/store through the ring of Section 9.2 and carries no packet header.

9.1 Memory registration

[R-9.1-002] MAP_SYNC SHALL be requested with MAP_SHARED_VALIDATE, never with MAP_SHARED. Since Linux 4.17 do_mmap() returns -EOPNOTSUPP for MAP_SHARED combined with any non-legacy flag, precisely to stop this usage, so v2.0's mmap(..., MAP_SHARED | MAP_SYNC, ...) **fails on every kernel it could run on**, DAX or not.

[R-9.1-003] If MAP_SHARED_VALIDATE | MAP_SYNC returns EOPNOTSUPP the implementation MAY fall back to MAP_SHARED and SHALL then clear UAF_CAP_SYNC_FAULT.

[R-9.1-004] Device-DAX alignment SHALL be honoured. A device-DAX instance commonly requires 2 MiB or 1 GiB alignment for both the mapping length and the file offset; a request that does not satisfy it SHALL fail with UAF_ERR_INVALID.

[R-9.1-005] uaf_reg_mr() on UAF-D SHALL succeed only for an address already inside a registered CXL window. Otherwise it SHALL return UAF_ERR_NOT_SUPPORTED, and the backend SHALL clear UAF_CAP_REG_ANY_MEM. v2.0 passed the caller's address to mmap as a hint without MAP_FIXED, which creates a new mapping somewhere else and registers that instead.

```
struct uaf_d_mr { uaf_mr_t pub; int dax_fd; uint64_t cxl_base; };

static int df_alloc_mr(uaf_domain_t *pd, size_t len, uint64_t flags,
                      uaf_mr_t **out)
{
    uint64_t align = df_dax_alignment(pd);          /* [R-9.1-004] */
    if (align == 0u || (len & (align - 1u))) return UAF_ERR_INVALID;

    int fd = open(pd->dax_path, O_RDWR | O_CLOEXEC);
    if (fd < 0) return UAF_ERR_NODEV;

    uint64_t off;
    int rc = df_alloc_cxl_window(pd, len, align, &off); /* checked */
    if (rc != UAF_OK) { close(fd); return rc; }

    int mflags = MAP_SHARED_VALIDATE | MAP_SYNC;      /* [R-9.1-002] */
    void *va = mmap(NULL, len, PROT_READ | PROT_WRITE, mflags, fd,
                    (off_t)off);
    if (va == MAP_FAILED && errno == EOPNOTSUPP) {
        va = mmap(NULL, len, PROT_READ | PROT_WRITE, MAP_SHARED, fd,
                  (off_t)off);
        if (va != MAP_FAILED) pd->caps &= ~UAF_CAP_SYNC_FAULT; /* [R-9.1-003] */
    }
    if (va == MAP_FAILED) {
        df_free_cxl_window(pd, off, len); close(fd); return UAF_ERR_NOMEM;
    }

    uint32_t lkey = 0, rkey = 0;
    rc = uaf_key_register(&pd->keys, (uint64_t)(uintptr_t)va, len, flags,
                        0u, &lkey, &rkey);          /* [R-11.2-001] */
    if (rc != UAF_OK) {
        munmap(va, len); df_free_cxl_window(pd, off, len); close(fd);
        return rc;
    }
    struct uaf_d_mr *m = calloc(1, sizeof(*m));
    if (!m) {
        uaf_key_deregister(&pd->keys, rkey); munmap(va, len);
        df_free_cxl_window(pd, off, len); close(fd); return UAF_ERR_NOMEM;
    }
    m->dax_fd = fd; m->cxl_base = off;
```

```

m->pub.struct_size = (uint32_t)sizeof(uaf_mr_t);
m->pub.pd = pd; m->pub.addr = va; m->pub.length = len;
m->pub.lkey = lkey; m->pub.rkey = rkey; m->pub.flags = flags;
*out = &m->pub;
return UAF_OK;
}

```

9.2 Send ring over CXL.mem

[R-9.2-001] The send ring index SHALL wrap on the **send ring's own depth**, which SHALL be a power of two. v2.0 wrapped with % UAF_CQ_DEPTH_DEFAULT (1024) a ring whose default depth was UAF_SQ_DEPTH_DEFAULT (256), writing up to 49152 bytes past the end for a 64-byte work request.

[R-9.2-002] Producer and consumer indices SHALL follow Section 10.3: the producer advances tail. v2.0's send path advanced head while its storage path advanced tail, in adjacent sections of one backend.

[R-9.2-003] UAF_DOORBELL_SQ and every other doorbell selector SHALL be defined in a public header. v2.0 used the identifier and defined it nowhere.

```

#define UAF_DOORBELL_SQ 0u
#define UAF_DOORBELL_RQ 1u
#define UAF_DOORBELL_CQ 2u

static int df_post_send(uaf_qp_t *qp, struct uaf_wr *wr)
{
    struct uaf_d_qp *d = uaf_d_qp(qp);
    int rc = df_validate_wr(d, wr);
    if (rc != UAF_OK) return rc;

    uint32_t slot;
    struct uaf_wr *e = uaf_ring_produce(&d->sq, &slot); /* [R-9.2-001] */
    if (!e) return UAF_ERR_BUSY;

    *e = *wr;
    memcpy(d->sge_pool + slot * d->max_sge, wr->sge,
           wr->num_sge * sizeof(struct uaf_sge));
    e->sge = d->sge_pool + slot * d->max_sge;
    uaf_ring_commit(&d->sq); /* [R-9.2-002] */

    df_ring_doorbell(qp, UAF_DOORBELL_SQ, d->sq.tail);
    return UAF_OK;
}

```

9.3 DST over a CXL-mapped NVMe BAR

```

static int df_storage_submit(uaf_sq_t *sq, struct uaf_storage_cmd *cmd)
{
    struct uaf_d_sq *s = uaf_d_sq(sq);
    if (cmd->struct_size != sizeof(*cmd)) return UAF_ERR_INVALID;

    int opc = uaf_dst_nvme_opcode(cmd->opcode); /* [R-6.2-001] */
    if (opc < 0) return opc;

    uint16_t nlb0 = 0;

```

```

if (opc != UAF_NVME_OPC_FLUSH) { /* [R-6.2-002] */
    int rc = uaf_dst_nlb_to_cdw12(cmd->nlb, &nlb0);
    if (rc != UAF_OK) return rc;
    if (cmd->buf_len - cmd->buf_offset <
        (uint64_t)cmd->nlb * s->block_size) return UAF_ERR_INVALID;
}

uint16_t cid;
int rc = uaf_cid_alloc(&s->cids, cmd->wr_id, &cid); /* [R-6.5-001] */
if (rc != UAF_OK) return rc;

uint32_t slot;
struct nvme_sq_entry *e = uaf_ring_produce(&s->ring, &slot);
if (!e) { uaf_cid_release(&s->cids, cid, NULL); return UAF_ERR_BUSY; }

memset(e, 0, sizeof(*e));
e->opcode = (uint8_t)opc;
e->cmd_id = cid;
e->nsid = s->nsid;
if (opc != UAF_NVME_OPC_FLUSH) {
    uint64_t iova = s->iova_base + cmd->buf_offset;
    enum uaf_dst_dptr kind;
    rc = uaf_dst_dptr_kind(iova, (uint64_t)cmd->nlb * s->block_size,
        s->page_size, s->sgl_supported, &kind);
    if (rc != UAF_OK) { uaf_cid_release(&s->cids, cid, NULL); return rc; }
    df_fill_dptr(e, kind, iova, cmd); /* [R-6.2-003] */
    e->cdw10 = (uint32_t)(cmd->lba & 0xFFFFFFFFFull);
    e->cdw11 = (uint32_t)(cmd->lba >> 32);
    e->cdw12 = nlb0;
}
uaf_ring_commit(&s->ring);

df_sq_clean_to_memory(s); /* [R-6.6-001] */
uaf_mmio_write32(s->sq_tdbl, s->ring.tail); /* [R-9.4-001] */
return UAF_OK;
}

```

9.4 Device-visible ordering

[R-9.4-001] A doorbell write SHALL be preceded by a barrier that orders prior descriptor stores against a store to device memory. `atomic_thread_fence(memory_order_seq_cst)` is **not** such a barrier: on AArch64 it compiles to `dmb ish`, which orders inner-shareable cacheable accesses and says nothing about Device-nGnRE. v2.0's "doorbell ordering fix" used exactly that fence on both software backends.

[R-9.4-002] The doorbell page SHALL be mapped as device memory (Device-nGnRE on AArch64, uncached or write-combining on x86).

[R-9.4-003] Producer and consumer indices in a ring shared with a device SHALL both be accessed atomically. v2.0's storage path stored `sq_tail` as a plain `uint32_t` while loading `sq_head` atomically.

```

static inline void uaf_dma_wmb(void)
{
    #if defined(__aarch64__) || defined(__arm__)
        __asm__ __volatile__ ("dsb st" ::: "memory");
    #elif defined(__x86_64__) || defined(__i386__)

```

```

__asm__ __volatile__ ("sfence" ::: "memory");
#else
    atomic_thread_fence(memory_order_seq_cst);
#endif
}

static inline void uaf_mmio_write32(volatile void *addr, uint32_t v)
{
    uaf_dma_wmb();
    *(volatile uint32_t *)addr = v;
}

```

10 State Machines, Memory, and Rings

10.1 Queue-pair state machine

[R-10.1-001] The legal transitions are exactly these. Every other transition SHALL fail with UAF_ERR_QP_STATE.

From	To	Trigger	Required attribute mask
RESET INIT	INIT RTR	modify_qp(INIT) modify_qp(RTR)	STATE, PORT STATE, RQ_PSN, DEST_QPN, PATH_MTU, MAX_DEST_RD_AT, and the address vector for the profile
RTR	RTS	modify_qp(RTS)	STATE, SQ_PSN, MAX_RD_ATOMIC, RETRY, RNR_RETRY, TIMEOUT
RTS SQD any	SQD RTS ERR	modify_qp(SQD) modify_qp(RTS) fault, or modify_qp(ERR)	STATE STATE STATE
any	RESET	modify_qp(RESET)	STATE

[R-10.1-002] The address-vector requirement is profile-dependent: Profile A requires AV_GID and AV_LID; Profile B requires AV_UDP. A CXL or UDP backend has neither a GID nor a LID, and v2.0's RTR rule demanded both from every backend.

[R-10.1-003] rq_psn is REQUIRED at RTR. v2.0's table never asked for it.

[R-10.1-004] UAF_QPS_SQE is entered by the implementation on a send-queue error and is left only through RESET or ERR. v2.0 declared the state in its enum and gave it no row in the table, so it was unreachable and undocumented.

[R-10.1-005] A work request already in flight when a fault occurs SHALL complete with the **originating** error — UAF_ERR_TIMEOUT, UAF_ERR_CRC, UAF_ERR_RKEY and so on. Only requests flushed *after* the transition complete with UAF_ERR_QP_STATE. v2.0 said every fault flushed with UAF_ERR_QP_STATE, which discarded the very status the rest of the document takes care to produce.

10.2 Key encoding

[R-10.2-001] One algorithm, every backend: lkey and rkey are CSPRNG values from the key table of Section 11.2, bound to a domain, a queue pair, a base, a length and a generation.

Backend	lkey	rkey	Address carrier
UAF-N	ibv_mr->lkey	ibv_mr->rkey	64-bit GPU or host virtual address
UAF-S	CSPRNG, key table	CSPRNG, key table, distinct from lkey	64-bit virtual address
UAF-D	CSPRNG, key table	CSPRNG, key table, distinct from lkey	64-bit CXL window offset

[R-10.2-002] On UAF-N the adapter owns the key space, so the verbs keys are authoritative and the software table is not used. A UAF-N implementation SHALL NOT synthesise its own keys.

v2.0's table described the UAF-D rkey as "32-bit random" while its code derived it from the CXL offset, length and flags, and described the UAF-S lkey as a descriptor number with the rkey a salted copy of it.

10.3 Ring discipline

[R-10.3-001] In every ring in this specification the **producer advances tail** and the **consumer advances head**, following NVMe.

[R-10.3-002] Every ring depth SHALL be a power of two and index wrap SHALL be a mask.

[R-10.3-003] The modulus SHALL be the depth of the ring being indexed. A constant naming a different queue MUST NOT appear in an index computation.

[R-10.3-004] Occupancy SHALL be computed as $\text{tail} - \text{head}$ in unsigned arithmetic, so a counter wrap at 2^{32} is well defined.

[R-10.3-005] Rings are single-producer, single-consumer ([R-4.9-001]).

11 Security

11.1 Threat model and mitigations

[R-11.1-001] The deployment model SHALL be stated in a conformance claim. Two are recognised: a **trusted fabric**, where every endpoint is administratively controlled, and an **untrusted fabric**, where it is not.

[R-11.1-002] On an untrusted fabric, link or network-layer authenticated encryption — PSP, IPsec ESP or MACsec — is REQUIRED, not optional. v2.0 listed them as optional with no key exchange, no SPI and no "required on the wire" rule, which left cleartext remote DMA behind an unauthenticated handshake as the default.

Threat	Mitigation	Requirement
Unauthorised remote write or read	CSPRNG rkey, bound to domain, QP, base, length and generation; bounds and access-flag check; IOMMU or SMMU	[R-11.2-001]..[R-11.2-005]
rkey brute force	Consecutive-failure limit, then the QP moves to ERR	[R-11.2-004]
Stale rkey after deregistration	Generation counter invalidates it	[R-11.2-005]
Payload corruption	hdr_crc32c and data_crc32c	[R-5.2-002], [R-6.3-002]
Packet injection or modification	PSP, IPsec ESP or MACsec. CRC32C does not mitigate this	[R-11.1-002], [R-11.1-003]
Connection hijack, CM spoofing	HMAC-SHA256 authenticator on every CM packet	[R-5.5-006]
CM replay	Nonce cache and timestamp window	[R-5.5-007]
Connection teardown by an off-path host	A malformed packet never moves a QP to ERR	[R-5.7-001]
Malicious DMA from a peripheral	Per-domain IOMMU or SMMU stage-1 tables	[R-11.1-004]
Physical CXL bus snooping	CXL IDE on UAF-D links	[R-11.1-005]
Confused deputy via a descriptor	No descriptor in any off-node structure	[R-5.5-005]

[R-11.1-003] CRC32C is an error-detecting code, not an integrity or authentication mechanism. An attacker who alters a packet recomputes it for free. v2.0's threat matrix named "Header CRC32C" as the mitigation for packet injection, which is a category error, and covered only header bytes 0..39 — leaving the payload, the CRC field itself and the reserved area outside any check.

[R-11.1-004] Every domain SHALL have its own IOMMU or SMMU stage-1 translation tables. A region SHALL NOT be reachable from a domain that did not register it.

[R-11.1-005] A UAF-D deployment carrying data outside a physically secured enclosure SHALL enable CXL Integrity and Data Encryption (IDE).

11.2 Key management

[R-11.2-001] lkey and rkey SHALL be generated by a CSPRNG. Neither SHALL be derived from an address, an offset, a length, a file descriptor, a counter, or any other value an attacker can guess or observe.

[R-11.2-002] lkey and rkey SHALL be distinct and non-zero. v2.0's UAF-D code set them equal, so disclosure of a local key conferred remote write.

[R-11.2-003] An rkey SHALL be bound to (domain, queue pair, base, length). A request presenting a valid rkey on a queue pair it was not issued for SHALL fail with UAF_ERR_RKEY.

[R-11.2-004] After UAF_RKEY_FAIL_MAX consecutive validation failures a queue pair SHALL move to UAF_QPS_ERR. A 32-bit key space is otherwise exhaustible in seconds: at 400 Gbps a 64-byte probe rate of order 10^9 per second covers 2^{32} keys in about four seconds.

[R-11.2-005] Deregistration SHALL increment the domain's generation counter so a stale rkey can never match a later registration.

[R-11.2-006] A successful validation SHALL reset the failure counter.

Conformance case C-5 exercises every clause above, including the brute-force limit and the stale-key case.

12 Performance and Conformance

12.1 Measurement method

[R-12.1-001] A performance figure in this specification is a **target**, not a measurement. A conformance claim SHALL report measured values obtained by the method below, and SHALL state the hardware, firmware and driver versions. v2.0 published a table with no percentile, no queue depth, no message size for the bandwidth rows, no statement of CPU versus GPU submission, and no pass/fail rule.

[R-12.1-002] The method SHALL be:

Parameter	Definition
Harness	uaf_perf from the conformance tree, one process per node
Latency metric	One-way, half of the round trip of a ping-pong, reported at p50, p99 and p99.9
Latency message sizes	64 B and 4096 B, reported separately
Latency queue depth	1
Bandwidth metric	Unidirectional goodput, payload bytes only, excluding the 64-byte header
Bandwidth message size	1 MiB
Bandwidth queue depth	64 outstanding messages per queue pair
IOPS metric	4 KiB random reads, queue depth 128 per queue, reported per device and per node
Warm-up	10 s discarded
Duration	60 s minimum
Scope	Stated as per queue pair, per device, or per node
Pass criterion	p99 latency within 20 % of target, bandwidth and IOPS within 10 %, zero errors and zero retransmissions on an idle fabric

12.2 Targets

[R-12.2-001] These targets replace v2.0's table. Each is accompanied by the ceiling it is derived from, so a reader can check it.

12.2.1 RMT latency (one-way, p50)

Path	64 B	4 KiB	Basis
UAF-N, IB RC	1.5 μ s	2.0 μ s	ConnectX-7/8 small-message RC; 4 KiB adds 82 ns of wire time at 400 Gbps
UAF-N, RoCEv2 RC	2.5 μ s	3.0 μ s	As above with Ethernet encapsulation
UAF-S, PCIe P2P or DSP	8 μ s	10 μ s	On-package path, no network stack

Path	64 B	4 KiB	Basis
UAF-S, UAF-RMT over UDP	20 μ s	25 μ s	Software UDP on an X Elite class core. v2.0 quoted 25 μ s without distinguishing this from the P2P path, and the platform has no RoCE offload
UAF-D, direct CXL.mem store	0.4 μ s	0.5 μ s	CXL 3.1 load-to-use of order 100–250 ns plus 34 ns to move 4 KiB at 128 GB/s
UAF-D, software ring (Section 9.2)	1.8 μ s	2.0 μ s	Adds a work-request copy, an MMIO doorbell and a responder poll

v2.0 listed 0.3 μ s for UAF-D without saying which path. The direct load/store path is close to that figure; the path the specification actually defines in Section 9.2 includes a ring, a work-request copy and a doorbell, and cannot reach it.

12.2.2 RMT bandwidth (unidirectional goodput, 1 MiB messages)

Path	Target	Link ceiling	Basis
UAF-N, ConnectX-7	380 Gbps	400 Gbps	95 % of line rate
UAF-N, ConnectX-8	760 Gbps	800 Gbps	95 % of line rate
UAF-S, PCIe P2P	64 Gbps	126 Gbps (PCIe 4.0 x8)	8 GB/s on the local path
UAF-S, UDP	25 Gbps	—	Software transport, multiple cores
UAF-D, CXL 3.1 x16	950 Gbps per direction	1024 Gbps per direction	64 GT/s $\times$ 16 lanes; 2048 Gbps is the duplex aggregate
UAF-D, CXL 4.0 x16 (OPTIONAL)	1900 Gbps per direction	2048 Gbps per direction	128 GT/s $\times$ 16 lanes, requires CXL 4.0

[R-12.2-002] A bandwidth figure SHALL state whether it is per direction or a duplex aggregate. v2.0's "2,000 Gbps peak unidirectional" for a CXL 3.1 link was the duplex aggregate of a PCIe 7.0 PHY quoted against a CXL 3.1 specification: two errors compounded.

12.2.3 DST throughput (4 KiB random read, QD 128)

Path	Per device	Per node	Basis
UAF-N	2.0 M IOPS per GPU	16 M IOPS, 8 GPUs	PCIe 5.0 x16 carries 13.4 M IOPS of 4 KiB traffic at 55 GB/s; 2.0 M is 15 % of that ceiling and is media-bound. Requires at least 8 PCIe 5.0 NVMe devices
UAF-S	1.2 M IOPS	1.2 M IOPS	A PCIe 4.0 x4 client NVMe device

Path	Per device	Per node	Basis
UAF-D	3.0 M IOPS	12 M IOPS, 4 devices	CXL-mapped NVMe; media-bound, not link-bound

[R-12.2-003] v2.0's "200 M+ IOPS" for UAF-N is withdrawn. At 4 KiB it implies $200 \times 10^6 \times 4096 = 819$ GB/s of storage traffic, roughly fifteen times a single PCIe 5.0 x16 link and beyond any published GPUDirect Storage result on one node. Likewise 50 M IOPS for UAF-D implies 205 GB/s from the storage media behind one accelerator.

12.3 Conformance suite

[R-12.3-001] An implementation SHALL pass every case below. The suite is in conformance/ and is built and run by conformance/run_conformance.sh, which configures with -DUAF_BUILD_TESTS=0N and runs ctest. v2.0's Section 12 declared UAF_BUILD_TESTS and then ended, with no test of any kind.

Case	Name	Covers	Requirements
C-1	abi	Struct sizes, offsets, CQE alignment and phase position, MTU accounting, no size_t or enum field	R-4.1-001..005, R-6.3-001
C-2	crc32c	RFC 3720 vectors and the golden header of Section 5.8	R-5.8-001
C-3	wire	Encode and decode round trip, big-endian order, bad magic, bad version, header CRC failure, non-zero reserved bytes and flags, UAF_WR_RECV rejected, CM and ACK/NAK opcodes, framing and segment accounting, payload CRC, atomic lengths, misaligned atomic, PSN wrap	R-5.2-002, R-5.3-001..007, R-5.4-001..004, R-5.6-001..003, R-5.7-001
C-4	qp_state	Legal and illegal transitions, SQE reachability, profile-dependent address vector, attribute-mask enforcement, attr->qp_state agreement, in-flight status preservation	R-4.5-003, R-4.5-004, R-10.1-001..005
C-5	rkey	Key generation, distinctness, QP binding, bounds, access flags, brute-force limit, generation invalidation	R-11.2-001..006

Case	Name	Covers	Requirements
C-6	ring	Power-of-two depth, full ring, wrap inside bounds across 4x depth, counter wrap at 2^{32} , independence of the two default depths	R-9.2-001, R-10.3-001..004
C-7	cqe_phase	Zeroed ring yields zero completions, batch bound by max, phase flip on wrap, stale entries not re-reported, failure status preserved, latency clamped	R-4.7-002, R-6.3-001..004
C-8	dst_map	Opcode map including flush, NLB 0 rejected, NLB 1 and 65536, PRP1/PRP2/PRP-list/SGL selection, cmd_id allocation and 64-bit wr_id round trip, table exhaustion	R-6.2-001..003, R-6.5-001..003
C-9	conn_wire	Connection record round trip, big-endian layout, host memcpy is not the wire form, no descriptor on the wire, reserved bytes, ambiguous profile, MTU floor	R-5.5-002..005
C-10	udp_loopback	An end-to-end multi-segment RDMA WRITE over UDP: segmentation, transmission, reassembly, one completion on LAST, byte-exact result, corrupted payload dropped without writing, wrong rkey refused, write past the region refused	R-5.3-003..005, R-5.7-001, R-11.2-003

[R-12.3-002] The suite SHALL build with -Wall -Wextra -Wpedantic -Werror.

13 Build System

```
cmake_minimum_required(VERSION 3.20)
project(uaf VERSION 2.1.0 LANGUAGES C)

set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
```

```

set(CMAKE_POSITION_INDEPENDENT_CODE ON)

option(UAF_BUILD_NVIDIA      "Build NVIDIA backend (UAF-N)"      OFF)
option(UAF_BUILD_SNAPDRAGON "Build Snapdragon backend (UAF-S)" OFF)
option(UAF_BUILD_DRAGONFLY  "Build Dragonfly backend (UAF-D)"  OFF)
option(UAF_BUILD_TESTS      "Build and register the conformance suite" ON)

add_library(uaf_core SHARED
    src/core/uaf_init.c      src/core/uaf_qp.c      src/core/uaf_cq.c
    src/core/uaf_mr.c        src/core/uaf_conn.c     src/core/uaf_storage.c
    src/core/uaf_wire.c      src/core/uaf_crc32c.c    src/core/uaf_ring.c
    src/core/uaf_rkey.c      src/core/uaf_dst.c      src/core/uaf_qp_state.c
)
target_include_directories(uaf_core PUBLIC include)

# A stable ABI needs a version on the shared object. v2.0 declared neither a
# SOVERSION nor an install rule for a library whose premise was ABI stability.
set_target_properties(uaf_core PROPERTIES
    VERSION    ${PROJECT_VERSION}
    SOVERSION  ${PROJECT_VERSION_MAJOR})

# One backend per process ([R-1.2-003]); each exports only uaf_pal_get_ops().
if(UAF_BUILD_NVIDIA)
    find_package(PkgConfig REQUIRED)
    pkg_check_modules(IBVERBS REQUIRED libibverbs)
    add_library(uaf_n SHARED src/pal/nvidia/uaf_pal_nvidia.c)
    target_link_libraries(uaf_n PRIVATE uaf_core ${IBVERBS_LIBRARIES} cufile)
endif()
if(UAF_BUILD_SNAPDRAGON)
    add_library(uaf_s SHARED src/pal/snapdragon/uaf_pal_snapdragon.c)
    target_link_libraries(uaf_s PRIVATE uaf_core uring)
endif()
if(UAF_BUILD_DRAGONFLY)
    add_library(uaf_d SHARED src/pal/dragonfly/uaf_pal_dragonfly.c)
    target_link_libraries(uaf_d PRIVATE uaf_core)
endif()

if(UAF_BUILD_TESTS)
    enable_testing()
    add_subdirectory(conformance)
endif()

include(GNUInstallDirs)
install(TARGETS uaf_core LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR})
install(DIRECTORY include/ DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}/uaf)

```

14 Appendices

14.1 Appendix A: Public header (uaf_core.h)

NORMATIVE.

```

#ifndef UAF_CORE_H
#define UAF_CORE_H

```

```

#include <stdint.h>
#include <stddef.h>
#include "uaf_config.h"
#include "uaf_types.h"

#ifdef __cplusplus
extern "C" {
#endif

/* Control functions return UAF_OK or a negative enum uaf_error.
 * Poll functions return a non-negative count or a negative error. */

int      uaf_init(void);
int      uaf_fini(void);
const char *uaf_version(void);
const char *uaf_backend_name(void);
uint32_t uaf_abi_version(void);

int uaf_get_device_list(uaf_device_t ***devs, int *count);
int uaf_free_device_list(uaf_device_t **devs, int count);
int uaf_query_device(uaf_device_t *dev, struct uaf_device_attr *attr);
int uaf_open_device(uaf_device_t *dev, uaf_domain_t **pd);
int uaf_close_device(uaf_device_t *dev);
int uaf_close_domain(uaf_domain_t *pd);

int uaf_reg_mr(uaf_domain_t *pd, void *addr, size_t len,
               uint64_t flags, uaf_mr_t **mr);
int uaf_alloc_mr(uaf_domain_t *pd, size_t len, uint64_t flags, uaf_mr_t **mr);
int uaf_dereg_mr(uaf_mr_t *mr);
int uaf_free_mr(uaf_mr_t *mr);

int uaf_create_cq(uaf_domain_t *pd, int cqe, uaf_cq_t **cq);
int uaf_destroy_cq(uaf_cq_t *cq);
int uaf_poll_cq(uaf_cq_t *cq, int max, struct uaf_wc *wc);

int uaf_create_qp(uaf_domain_t *pd, uaf_cq_t *scq, uaf_cq_t *rcq,
                  struct uaf_qp_init_attr *init, uaf_qp_t **qp);
int uaf_modify_qp(uaf_qp_t *qp, enum uaf_qp_state state,
                  struct uaf_qp_attr *attr, uint32_t attr_mask);
int uaf_destroy_qp(uaf_qp_t *qp);
int uaf_post_send(uaf_qp_t *qp, struct uaf_wr *wr);
int uaf_post_recv(uaf_qp_t *qp, struct uaf_wr *wr);

int uaf_get_conn_info(uaf_qp_t *qp, struct uaf_conn_info *local);
int uaf_connect(uaf_qp_t *qp, const struct uaf_conn_info *remote);
int uaf_disconnect(uaf_qp_t *qp);
int uaf_conn_info_serialize(const struct uaf_conn_info *ci, uint8_t out[64]);
int uaf_conn_info_deserialize(const uint8_t in[64], struct uaf_conn_info *ci);

int uaf_map_peer(uaf_domain_t *pd, const struct uaf_peer_info *peer,
                 uaf_mr_t **mr);
int uaf_unmap_peer(uaf_mr_t *mr);

int uaf_storage_open(uaf_domain_t *pd, const char *path,
                     uaf_sq_t **sq, uaf_cr_t **cr);
int uaf_storage_submit(uaf_sq_t *sq, struct uaf_storage_cmd *cmd);
int uaf_storage_poll(uaf_cr_t *cr, int max, struct uaf_storage_cqe *cqe);
int uaf_storage_close(uaf_sq_t *sq, uaf_cr_t *cr);

```

```

int uaf_storage_reset(uaf_sq_t *sq, uaf_cr_t *cr);

const char *uaf_strerror(int e);
void        uaf_set_log_level(int level);
void        uaf_set_log_callback(void (*cb)(int level, const char *msg));

#ifdef __cplusplus
}
#endif
#endif /* UAF_CORE_H */

```

14.2 Appendix B: Reference application (rdma_write_demo.c)

INFORMATIVE, but every return value is checked and the connection is established. v2.0's Appendix B never called `uaf_connect()`, wrote to remote address 0 with rkey 0 from a zeroed `uaf_conn_info`, then polled forever for a completion that could not arrive, and ignored twelve return values.

```

#include "uaf_core.h"
#include <stdio.h>
#include <string.h>

#define CK(expr) do { int _rc = (expr); if (_rc != UAF_OK) { \
    fprintf(stderr, "%s failed: %s\n", #expr, uaf_strerror(_rc)); \
    return 1; } } while (0)

/* Exchanges 64 serialised bytes with the peer; implemented by the caller
 * over TCP, a job launcher, or any out-of-band channel. */
extern int exchange_conn(const uint8_t out[64], uint8_t in[64]);

int main(void)
{
    CK(uaf_init());

    uaf_device_t **devs = NULL;
    int n = 0;
    CK(uaf_get_device_list(&devs, &n));
    if (n == 0) { fprintf(stderr, "no UAF device\n"); return 1; }

    struct uaf_device_attr da = { .struct_size = sizeof(da) };
    CK(uaf_query_device(devs[0], &da));
    if (!(da.caps & UAF_CAP_ATOMICS))
        printf("note: atomics unavailable on this backend\n");

    uaf_domain_t *pd = NULL;
    CK(uaf_open_device(devs[0], &pd));

    uaf_cq_t *cq = NULL;
    CK(uaf_create_cq(pd, 64, &cq));

    struct uaf_qp_init_attr init = {
        .struct_size = sizeof(init),
        .qp_type = UAF_QPT_RC,
        .max_send_wr = 64, /* powers of two */
        .max_recv_wr = 64,
        .max_send_sge = 1,
        .max_recv_sge = 1,
    };

```

```

        .max_inline_data = da.max_inline_data,
        .sq_sig_all      = 0,
    };
    uaf_qp_t *qp = NULL;
    CK(uaf_create_qp(pd, cq, cq, &init, &qp));

    /* Always-portable buffer acquisition: the library owns the allocation,
     * so this works whether or not UAF_CAP_REG_ANY_MEM is set. */
    uaf_mr_t *mr = NULL;
    CK(uaf_alloc_mr(pd, 4096,
                    UAF_MR_LOCAL_WRITE | UAF_MR_REMOTE_WRITE |
                    UAF_MR_REMOTE_READ, &mr));
    memset(mr->addr, 0, mr->length);
    snprintf((char *)mr->addr, mr->length, "Hello, UAF v2.1!");
    const uint32_t payload_len = (uint32_t)strlen((char *)mr->addr) + 1u;

    /* Exchange connection information, then connect. */
    struct uaf_conn_info local = { .struct_size = sizeof(local) };
    CK(uaf_get_conn_info(qp, &local));
    local.rkey = mr->rkey;
    local.remote_va = (uint64_t)(uintptr_t)mr->addr;

    uint8_t tx[64], rx[64];
    CK(uaf_conn_info_serialize(&local, tx));
    if (exchange_conn(tx, rx) != 0) { fprintf(stderr, "exchange failed\n"); return 1; }

    struct uaf_conn_info remote;
    CK(uaf_conn_info_deserialize(rx, &remote));
    CK(uaf_connect(qp, &remote)); /* drives INIT -> RTR -> RTS */

    struct uaf_sge sge = {
        .addr = (uint64_t)(uintptr_t)mr->addr,
        .length = payload_len, /* not the whole 4 KiB region */
        .lkey = mr->lkey,
    };
    struct uaf_wr wr = {
        .wr_id = 1,
        .opcode = UAF_WR_RDMA_WRITE,
        .num_sge = 1,
        .sge = &sge,
        .send_flags = UAF_SEND_SIGNED,
        .remote_addr = remote.remote_va,
        .rkey = remote.rkey,
    };
    CK(uaf_post_send(qp, &wr));

    /* Poll returns a COUNT: 0 means empty, negative means failure. */
    struct uaf_wc wc;
    for (;;) {
        int got = uaf_poll_cq(cq, 1, &wc);
        if (got < 0) {
            fprintf(stderr, "poll_cq: %s\n", uaf_strerror(got));
            return 1;
        }
        if (got == 1) break;
    }
    if (wc.status != UAF_OK) {
        fprintf(stderr, "write failed: %s\n", uaf_strerror(wc.status));
    }

```

```

    return 1;
}
printf("write completed: %u bytes\n", wc.byte_len);

CK(uaf_disconnect(qp));
CK(uaf_free_mr(mr));
CK(uaf_destroy_qp(qp));
CK(uaf_destroy_cq(cq));
CK(uaf_close_domain(pd));           /* absent from the v2.0 ABI */
CK(uaf_close_device(devs[0]));
CK(uaf_free_device_list(devs, n));
CK(uaf_fini());
return 0;
}

```

14.3 Appendix C: Requirement index

Requirement	Defined in
[R-1.2-001]	Non-goals
[R-1.2-002]	Operating environment
[R-1.2-003]	Operating environment
[R-1.4-001]	Normative language
[R-1.4-002]	Normative language
[R-1.4-003]	Normative language
[R-1.6-001]	Approval record
[R-3.2-001]	Wire profiles
[R-3.2-002]	Wire profiles
[R-3.2-003]	Wire profiles
[R-3.2-004]	Wire profiles
[R-3.2-005]	Wire profiles
[R-3.3-001]	CXL generation
[R-3.3-002]	CXL generation
[R-3.4-001]	Compile-time configuration (uaf_config.h)
[R-3.4-002]	Compile-time configuration (uaf_config.h)
[R-4.1-001]	ABI rules
[R-4.1-002]	ABI rules
[R-4.1-003]	ABI rules
[R-4.1-004]	ABI rules
[R-4.1-005]	ABI rules
[R-4.4-001]	Work requests and completions
[R-4.4-002]	Work requests and completions
[R-4.4-003]	Work requests and completions
[R-4.4-004]	Work requests and completions
[R-4.5-001]	Queue pair creation and modification
[R-4.5-002]	Queue pair creation and modification
[R-4.5-003]	Queue pair creation and modification
[R-4.5-004]	Queue pair creation and modification
[R-4.6-001]	Memory registration
[R-4.6-002]	Memory registration
[R-4.6-003]	Memory registration
[R-4.6-004]	Memory registration
[R-4.7-001]	Return conventions
[R-4.7-002]	Return conventions
[R-4.7-003]	Return conventions
[R-4.7-004]	Return conventions
[R-4.8-001]	Device capability query

Requirement	Defined in
[R-4.8-002]	Device capability query
[R-4.9-001]	Threading
[R-4.9-002]	Threading
[R-4.9-003]	Threading
[R-4.10-001]	Backend loading
[R-4.10-002]	Backend loading
[R-4.10-003]	Backend loading
[R-5.1-001]	RMT Protocol — Profile B (UAF-RMT over UDP)
[R-5.1-002]	Packet format
[R-5.2-001]	Field definitions
[R-5.2-002]	Field definitions
[R-5.3-001]	Opcode semantics
[R-5.3-002]	Opcode semantics
[R-5.3-003]	Framing
[R-5.3-004]	Framing
[R-5.3-005]	Framing
[R-5.3-006]	Framing
[R-5.3-007]	Framing
[R-5.4-001]	Atomics
[R-5.4-002]	Atomics
[R-5.4-003]	Atomics
[R-5.4-004]	Atomics
[R-5.4-005]	Atomics
[R-5.4-006]	CXL generation
[R-5.4-007]	Atomics
[R-5.5-001]	Connection management
[R-5.5-002]	Connection record (64 bytes, big-endian)
[R-5.5-003]	Connection record (64 bytes, big-endian)
[R-5.5-004]	Connection record (64 bytes, big-endian)
[R-5.5-005]	Connection record (64 bytes, big-endian)
[R-5.5-006]	Authenticator (32 bytes)
[R-5.5-007]	Authenticator (32 bytes)
[R-5.5-008]	Handshake
[R-5.5-009]	Handshake
[R-5.5-010]	Handshake
[R-5.5-011]	Handshake
[R-5.6-001]	Reliability
[R-5.6-002]	Reliability
[R-5.6-003]	Reliability
[R-5.6-004]	Reliability
[R-5.6-005]	Reliability
[R-5.6-006]	Reliability
[R-5.6-007]	Reliability
[R-5.7-001]	Error handling
[R-5.8-001]	Test vectors
[R-5.9-001]	Congestion control
[R-5.9-002]	Congestion control
[R-5.9-003]	Congestion control
[R-5.9-004]	Congestion control
[R-6.1-001]	Structure of this section
[R-6.1-002]	Structure of this section
[R-6.2-001]	Command mapping
[R-6.2-002]	Command mapping
[R-6.2-003]	Command mapping
[R-6.2-004]	Command mapping
[R-6.3-001]	Completion ring
[R-6.3-002]	Completion ring

Requirement	Defined in
[R-6.3-003]	Completion ring
[R-6.3-004]	Completion ring
[R-6.4-001]	Storage queue state machine
[R-6.4-002]	Storage queue state machine
[R-6.5-001]	Command identifiers
[R-6.5-002]	Command identifiers
[R-6.5-003]	Command identifiers
[R-6.6-001]	Submission ring memory type
[R-6.6-002]	Submission ring memory type
[R-7.1-001]	Profile and capabilities
[R-7.2-001]	Device discovery
[R-7.3-001]	Profile and capabilities
[R-7.3-002]	Memory registration
[R-7.3-003]	Memory registration
[R-7.4-001]	DST via cuFile
[R-7.4-002]	DST via cuFile
[R-8.1-001]	Platform Backend: UAF-S (Snapdragon, Linux)
[R-8.1-002]	Platform Backend: UAF-S (Snapdragon, Linux)
[R-8.2-001]	Memory registration
[R-8.2-002]	Memory registration
[R-8.2-003]	Memory registration
[R-8.2-004]	Memory registration
[R-8.2-005]	Memory registration
[R-8.3-001]	Send path
[R-8.3-002]	Send path
[R-8.3-003]	Send path
[R-9.1-001]	Platform Backend: UAF-D (Dragonfly)
[R-9.1-002]	Memory registration
[R-9.1-003]	Memory registration
[R-9.1-004]	Memory registration
[R-9.1-005]	Memory registration
[R-9.2-001]	Send ring over CXL.mem
[R-9.2-002]	Send ring over CXL.mem
[R-9.2-003]	Send ring over CXL.mem
[R-9.4-001]	Send path
[R-9.4-002]	Device-visible ordering
[R-9.4-003]	Device-visible ordering
[R-10.1-001]	Queue-pair state machine
[R-10.1-002]	Queue-pair state machine
[R-10.1-003]	Queue-pair state machine
[R-10.1-004]	Queue-pair state machine
[R-10.1-005]	Queue-pair state machine
[R-10.2-001]	Key encoding
[R-10.2-002]	Key encoding
[R-10.3-001]	Ring discipline
[R-10.3-002]	Ring discipline
[R-10.3-003]	Ring discipline
[R-10.3-004]	Ring discipline
[R-10.3-005]	Ring discipline
[R-11.1-001]	Threat model and mitigations
[R-11.1-002]	Threat model and mitigations
[R-11.1-003]	Threat model and mitigations
[R-11.1-004]	Threat model and mitigations
[R-11.1-005]	Threat model and mitigations
[R-11.2-001]	Memory registration
[R-11.2-002]	Key management
[R-11.2-003]	Key management

Requirement	Defined in
[R-11.2-004]	Threat model and mitigations
[R-11.2-005]	Threat model and mitigations
[R-11.2-006]	Key management
[R-12.1-001]	Measurement method
[R-12.1-002]	Measurement method
[R-12.2-001]	Targets
[R-12.2-002]	RMT bandwidth (unidirectional goodput, 1 MiB messages)
[R-12.2-003]	DST throughput (4 KiB random read, QD 128)
[R-12.3-001]	Conformance suite
[R-12.3-002]	Conformance suite

Total: 162 numbered requirements.

14.4 Appendix D: Open issues

These are carried deliberately. [R-1.6-001] requires each to be closed or accepted before this document may be marked approved.

ID	Issue	Disposition sought
OI-1	No asynchronous event channel: an application learns of link-down or a queue-pair fatal only by polling.	Add <code>uaf_get_async_event()</code> in v2.2, or accept polling for v2.1.
OI-2	DOCA GPUNetIO is unmapped. A UAF-N implementation could use it to speak Profile B, at the cost of <code>UAF_CAP_HW_CC</code> .	Specify the binding, or state that UAF-N is Profile A only.
OI-3	Interrupt-driven completion (<code>UAF_CAP_CQ_EVENTS</code>) is declared but has no API. Polling costs a core, which matters on Snapdragon.	Add the event API in v2.2.
OI-4	<code>UAF_QPT_UD</code> is declared and unspecified.	Specify it, or remove the enumerator.
OI-5	Multi-producer posting (<code>UAF_CAP_MT_POST</code>) is referenced by [R-4.9-001] and has no defined implementation.	Specify the compare-and-swap ring discipline, or withdraw the capability.
OI-6	The CM pre-shared fabric key has no distribution mechanism.	Name an external key-management dependency.
OI-7	Performance targets are unvalidated on silicon.	Run the method of Section 12.1 and replace targets with measurements.
OI-8	A Windows binding for UAF-S (DirectStorage) is deferred to UAF-SPEC-002.	Schedule it, or drop Windows from the roadmap.

14.5 Appendix E: Disposition of the v2.0 reviews

Both reviews are dated 2026-09-29. “R1” is the first, “R2” the second.

Finding	Source	Disposition
One wire format cannot describe three transports	R1, R2	Fixed: Section 3.2, two declared profiles; principle 5 rewritten
poll_cq return convention undefined	R1, R2	Fixed: [R-4.7-001]..[R-4.7-004], case C-7
No message framing; multi-packet messages inexpressible	R1	Fixed: Section 5.3, cases C-3 and C-10
Atomics unimplementable; no response packet	R1, R2	Fixed: Section 5.4, case C-3
DST CQE has no phase bit	R1, R2	Fixed: Section 6.3, case C-7
MAP_SHARED \ MAP_SYNC cannot succeed	R1	Fixed: [R-9.1-002], [R-9.1-003]
Send ring wrapped on the wrong depth constant	R1, R2	Fixed: [R-9.2-001], case C-6
ibv_device use-after-free	R1	Fixed: [R-7.2-001]
nlb == 0 underflow	R1, R2	Fixed: [R-6.2-002], case C-8
UAF_WR_STORAGE_FLUSH issued as a write	R2	Fixed: [R-6.2-001], case C-8
UAF-S registration copies and relocates	R1, R2	Fixed: Section 4.6, [R-8.2-001], [R-8.2-002]
Key model contradicts the threat matrix	R1, R2	Fixed: Section 11.2, case C-5
dma_buf_fd in a connection record	R1, R2	Fixed: [R-5.5-005], case C-9
CRC32C miscast as an injection defence	R1, R2	Fixed: [R-11.1-003]
CM unspecified; no layout, auth, teardown or simultaneous open	R2	Fixed: Section 5.5
CM and ACK/NAK opcodes in no enum	R1, R2	Fixed: enum uaf_wire_opcode, case C-3
UAF_WR_RECV listed as a wire opcode	R2	Fixed: [R-5.3-002], case C-3
Reliability parameters inconsistent with the latency targets	R1, R2	Fixed: Section 5.6
DCQCN underspecified	R1, R2	Fixed: Section 5.9
create_qp has no depth, SGE or type; modify_qp has no mask	R2	Fixed: Section 4.5, case C-4
RTR demands GID and LID from backends without them	R2	Fixed: [R-10.1-002], case C-4
UAF_QPS_SQE unreachable	R1, R2	Fixed: [R-10.1-004], case C-4
path_mtu unit undefined	R1, R2	Fixed: [R-4.5-002]
Every fault flushes as UAF_ERR_QP_STATE	R2	Fixed: [R-10.1-005], case C-4
enum width and size_t in the ABI	R1, R2	Fixed: [R-4.1-001], case C-1
No struct size or version field	R2	Fixed: [R-4.1-003]
uaf_mr.pal_priv in the application ABI	R1, R2	Fixed: [R-4.1-005], case C-1
packed CQE drops alignment to 1	R1	Fixed: [R-6.3-001], case C-1
No uaf_query_device	R1, R2	Fixed: Section 4.8
No domain destructor	R1	Fixed: uaf_close_domain()
storage_close releases only the submission queue	R1, R2	Fixed: [R-6.4-002]
No storage_reset for the ERROR edge	R1	Fixed: [R-6.4-001]

Finding	Source	Disposition
Section 6.1 pseudo-SQE never used	R1, R2	Fixed: deleted; Section 6.2 is an NVMe mapping
cmd_id truncates wr_id	R1, R2	Fixed: Section 6.5, case C-8
Single PRP cannot describe a multi-page buffer	R2	Fixed: [R-6.2-003], case C-8
PAL linking unspecified; all three symbols in one header	R2	Fixed: Section 4.10
Threading unspecified	R1, R2	Fixed: Section 4.9
WR and SGE ownership at post time unspecified	R2	Fixed: [R-4.4-002]
SGE list cannot be expressed on the wire	R1, R2	Fixed: [R-4.4-004]
imm_data validity unsignalled	R1	Fixed: [R-4.4-001]
rn timer has no behaviour	R2	Fixed: [R-5.6-006]
Barrier class wrong for AArch64 MMIO	R1, R2	Fixed: Section 9.4
Submission-queue memory type unstated	R2	Fixed: [R-6.6-001]
UAF_D00RBELL_SQ undefined	R2	Fixed: [R-9.2-003]
Device-DAX alignment unstated	R2	Fixed: [R-9.1-004]
“SQ” means two different queues	R2	Fixed: STQ in Section 2; Sections 9.2 and 9.3 disambiguated
remote_register_buf_attr truncates size_t	R2	Fixed: [R-8.2-004]
sd_post_send may submit twice	R2	Fixed: [R-8.3-001]
Non-coherent buffer never synchronised	R1, R2	Fixed: [R-8.2-003]
ibv_reg_mr failures all become UAF_ERR_NOMEM	R2	Fixed: [R-7.3-003]
UAF_MR_GPU ignored; no dma-buf path	R1, R2	Fixed: [R-7.3-001], [R-7.3-002]
cuFile path incomplete	R2	Fixed: [R-7.4-001]
MTU excludes the header	R1	Fixed: Section 3.4
CXL 3.1 paired with PCIe Gen7	R2	Fixed: [R-3.3-001]
UAF_ENABLE_CXL_CACHE off while CXL.cache advertised	R2	Fixed: [R-3.3-002]
CXL bandwidth double-counts the duplex link	R1, R2	Fixed: [R-12.2-002]
200 M IOPS and 0.3 µs not credible	R1, R2	Fixed: [R-12.2-001], [R-12.2-003]
Performance table has no method	R1, R2	Fixed: Section 12.1
No conformance surface, no requirement IDs, no SHALL	R1, R2	Fixed: Section 1.4, Section 12.3, Appendix C
No approver or open-issues list	R2	Fixed: Section 1.6, Appendix D
Windows and Linux in one backend	R2	Fixed: [R-1.2-001], [R-8.1-002]
Non-goals and OS matrix absent	R2	Fixed: Section 1.2
Reference application non-functional	R1, R2	Fixed: Appendix B
32-bit PSN versus IB’s 24-bit	R1	Fixed: [R-5.6-002], case C-3
Payload outside every integrity check	R1, R2	Fixed: data_crc32c, [R-5.2-002]
Bad magic or CRC could move a QP to ERR	R2	Fixed: [R-5.7-001], case C-10